# Intelligent DDoS Detection and Mitigation Framework for SDN Networks

Project report — Group 9


| Register No. | Name |
|---|---|
| 25BCE0507 | Ayush Biswas |
| 25BAI0022 | Ustat Kaur |
| 25BCE2096 | Gali Hasini |


Stack: Mininet + Open vSwitch + os-ken (Ryu) + Python ML

Data: CICDDoS2019 and controlled Mininet attack/benign traces

October 2026

# Abstract

Software-defined networks let a controller see per-flow counters from every switch and install drop rules within milliseconds, which makes them a natural place to detect and stop DDoS floods. We built and experimentally validated such a defense on Mininet, Open vSwitch and os-ken (the maintained fork of Ryu). Every 2 s the controller turns OpenFlow flow statistics into ten per-source features, scores them with seven detectors running side by side, lets one detector drive drop-rule mitigation with a quiet-based unblock policy, and records a SHAP explanation for every block. We evaluated it in 447 live emulation runs: SYN, UDP and ICMP floods at 50, 500 and 5 000 packets/s per attacker, under different benign loads, link bandwidths, delay and loss, and topology sizes, with five seed-paired repetitions per condition. The proposed XGBoost detector, chosen by a rule fixed before evaluation, reached F1 = 0.998 [0.996, 0.999] with a 0.03 % false-positive rate on held-out runs. A static pps threshold tuned on the same training data reached F1 = 0.570 and falsely flagged 32.51 % of benign intervals. In the closed loop, across the nine core attack conditions, benign TCP goodput during attacks averaged 1.00 of the offered rate with XGBoost, against 0.62 with no defense and 0.36 with the threshold, and attackers were blocked 3.19 s after starting on average. The gain over simpler trained models was small: a decision tree reached F1 = 0.994 and was statistically indistinguishable from XGBoost on every closed-loop metric, and a random forest was marginally better than XGBoost (0.998). On CICDDoS2019, restricting detectors to the features an OpenFlow switch can report cut XGBoost's F1 from 0.997 to 0.850 when training and test days differ. Code, raw logs and seeds for every number are in the repository.



*Keywords:* software-defined networking, DDoS, OpenFlow, Mininet, os-ken, XGBoost, SHAP, CICDDoS2019.

Abstract.....
1. Introduction.....
2. Background and related work.....
3. System architecture.....
4. Detection and mitigation design.....
5. Implementation.....
6. Datasets.....
7. Experimental setup.....
8. Results.....
9. Discussion.....
10. Conclusion and future work.....
References.....
Appendix A. Reproduction and repository structure.....
Appendix B. Parameters.....
Appendix C. Additional results.....

# 1. Introduction

## 1.1 Problem

A distributed denial-of-service (DDoS) attack floods a victim with traffic from many sources until legitimate users can no longer be served. Software-defined networking (SDN) changes both sides of this problem. It helps the defender, because a logically centralised controller sees per-flow counters from every switch and can install a drop rule anywhere in the network within milliseconds. It also creates new weak points: the controller and the switch-to-controller channel can themselves be saturated, and flow tables can be exhausted [6]. Detection inside the controller must therefore be accurate, fast and cheap.

Most published SDN DDoS detectors are evaluated offline, on public flow datasets such as CICDDoS2019 [11], with ~80 CICFlowMeter features and random train/test splits. Three gaps follow. First, an OpenFlow 1.3 switch cannot compute most of those features, so the reported accuracy is not what a controller would see. Second, random row-level splits leak information between windows of the same capture, which inflates accuracy. Third, detection accuracy alone says little about what users experience: a detector must be judged by how quickly it removes the attack and by how much legitimate traffic it harms on the way.

## 1.2 Objectives

1. Build a working SDN defense on Mininet [12], Open vSwitch and os-ken [16] (the maintained fork of Ryu [15]) that detects DDoS sources from OpenFlow flow statistics only, mitigates them with drop rules, and lifts the block when the source goes quiet.
2. Compare machine-learning detectors (random forest [20], XGBoost [19], an autoencoder) with conventional baselines (no defense, a tuned static rate threshold, a decision tree, logistic regression and an SVM) on **identical live traffic**, under several controlled network conditions, with repeated runs and confidence intervals.
3. Measure what matters to users: legitimate throughput, latency and HTTP success during an attack, time to mitigation, false blocks, and controller overhead.
4. Explain every block decision with SHAP feature attributions [17], [18], and use CICDDoS2019 to quantify how much accuracy is lost when a detector is restricted to the features an OpenFlow switch can actually provide.

## 1.3 Contributions

- **A reproducible testbed.** One Docker image and one command per phase reproduce every run; each of the 447 runs keeps its raw logs, seed and ground truth (Appendix A).
- **Shadow evaluation.** Every detector scores every polling interval of every run, while only one detector drives mitigation. Detection metrics are therefore paired on identical traffic, and mitigation runs of different modes share seeds.
- **Controller-feasible features with correct counter handling.** The original prototype lost traffic whenever a flow expired; the new flow-delta tracker recovers it from FlowRemoved messages (Section 5.3).
- **An honest comparison.** The proposed detector (XGBoost, selected by a rule fixed before evaluation) is compared with a *tuned* threshold and with a decision tree. Section 8 reports where it wins, where the improvement is small, and where it does not win.

- **A critical review** of the original repository (REVIEW.md, condensed in Section 9.3), with every fix listed in CHANGELOG.md.

### 1.4 Report structure

Section 2 gives background and reviews related work. Section 3 presents the architecture and Section 4 the detection and mitigation logic. Section 5 covers the implementation, Section 6 the datasets and Section 7 the experimental setup. Section 8 reports all results, Section 9 discusses them together with threats to validity, and Section 10 concludes.

## 2. Background and related work

### 2.1 SDN and OpenFlow

SDN separates the control plane (routing decisions) from the data plane (packet forwarding). In OpenFlow [13], [14] a switch holds flow tables of match–action rules; a packet that matches no rule is sent to the controller as a *packet-in*, and the controller answers by installing a rule (*flow-mod*). Each rule keeps cumulative packet and byte counters plus its age, which the controller can read with a flow-statistics request. When a rule expires, the switch can report its final counters in a *flow-removed* message. These counters are the only traffic measurements available to our detector; OpenFlow 1.3 has no per-packet inter-arrival times, TCP flags or payload statistics.

### 2.2 DDoS attacks considered

We study three volumetric floods that are standard in the SDN literature [6]: **TCP SYN floods** (half-open connections, small packets, exhausting connection state and packet-processing capacity), **UDP floods** (large datagrams to a closed port, exhausting bandwidth) and **ICMP floods** (echo requests, exhausting bandwidth and forcing replies). Each is generated at three rates per attacker (low 50 pps, medium 500 pps, high 5 000 pps) from several hosts at once. Low-rate floods are deliberately included: at 50 pps an attacker sends fewer packets per second than a benign file transfer, and this is where rate-based detection is expected to fail. Reflection/amplification attacks, which dominate CICDDoS2019, and spoofed-source floods are outside the scope of the live experiments (Section 9.2).

### 2.3 Machine learning for DDoS detection in SDN

Braga et al. [25] established the standard pattern: poll flow statistics from switches through the controller, aggregate them into a small feature vector, and classify it (with self-organising maps in their case). Tan et al. [26] combine a data-plane trigger with K-Means and KNN on rate and asymmetry features, and Ahuja et al. [27] train a hybrid SVC–random-forest classifier on SDN flow features. The recent survey by Wang et al. [6] organises this work into an attack taxonomy (including controller saturation and flow-table exhaustion), detection families and the limits of current countermeasures. Our design follows the same controller-polled pattern, but evaluates it in a closed loop rather than offline.

## 2.4 Recent work: federated, explainable and collaborative detection

The ten 2025–2026 references assigned to the project fall into three groups. Table entries summarise each paper from its abstract; claims and numbers are the authors' own, not reproduced here.

**Table 1.** Assigned 2025–2026 references compared with this project. 'Expl.' = explainability method; 'FL' = federated learning.

| Ref. | Focus | SDN / DDoS | FL | Expl. | Relation to this project |
|---|---|---|---|---|---|
| [4] Ma & Su | Autoencoder-MLP detector trained with FL and secure aggregation; rate-limiting push-back on P4 switches | Yes / Yes | Yes | — | Closest in scope (SDN DDoS detection + mitigation). We use one controller and drop rules, and add per-decision SHAP. |
| [6] Wang et al. | Survey of DDoS detection and defense in SDN | Yes / Yes | — | — | Threat model and taxonomy for Sections 2.2 and 9. |
| [2] Kalakoti et al. | FedXAI: LSTM IDS with FL; client SHAP aggregated securely; SHAP compared with LIME and IG | No / No | Yes | SHAP | Supports choosing SHAP; reports SHAP best on faithfulness and robustness. |
| [3] Sahoo et al. | Choir-IDS: federated neural boosting ensembles with calibrated probabilities; LIME + SHAP | No / No | Yes | SHAP, LIME | Motivates calibrated scores alongside attributions. |
| [1] Carillo et al. | Federated class-incremental learning for encrypted traffic classification, with XAI | No / No | Yes | Attribution | Relevant to future work on new attack classes. |
| [5] Tellache et al. | Federated multi-agent RL NIDS (one agent per attack type) on CIC-IDS-2017 | No / No | Yes | — | Collaborative detection across domains; future work. |
| [7] Cajaraville-Aboy et al. | CO-DEFEND: decentralised FL for DoH tunnelling; DT/RF kept interpretable by model selection | No / No | Yes | Inherent (trees) | Shows how tree ensembles like ours could be federated. |
| [8] Li et al. | CANDICE: integrated detector + counterfactual explainer producing intrusion profiles | No / No | — | Counterfactual | Critique of post-hoc attribution faithfulness; contrasts with our SHAP. |
| [9] Minh et al. | Explainable stacking ensemble of unsupervised detectors on short traffic windows | No / No | — | Heatmaps | Closest feature design: window-aggregated statistics, operator-facing explanations. |
| [10] Houssel et al. | eX-NIDS: LLM explanations of flagged flows with context-augmented prompts | No / No | — | LLM text | Future work: turn SHAP attributions into operator-readable text. |

Two observations shape our design. First, only [4] and [6] are both SDN- and DDoS-specific; the remaining eight address intrusion or traffic classification in general, mostly in IoT or encrypted-traffic settings. Second, explainability in this literature is either post-hoc attribution (SHAP, LIME, IG in [2], [3]) or built into the model ([7]–[10]). We adopt exact TreeSHAP [18] because it is cheap for tree ensembles, which lets the controller attach an explanation to *every* block decision online, not only to offline analyses. The federated and multi-agent directions of [1]–[5], [7] are complementary to our single-controller setting; we return to them in Section 10.

## 2.5 Datasets and their pitfalls

CICDDoS2019 [11] contains two capture days (12 January and 11 March in the file names) of reflection and exploitation attacks, with flows extracted by CICFlowMeter. Independent audits found labelling and feature-extraction errors in CICIDS2017, which was produced with the same tool chain [22]–[24]; we found no peer-reviewed audit of CICDDoS2019 itself, so these critiques

apply by shared tooling rather than by direct evidence. Two properties matter for us: benign flows are a small minority (Section 6.1), and most of the ~80 features are bidirectional statistics that an OpenFlow switch cannot produce.

# 3. System architecture

Figure 1 shows the system. The data plane is a Mininet network of Open vSwitch switches connecting benign hosts, attackers and one server. The control plane is a single os-ken application. An offline pipeline turns the logs of every run into labelled datasets, trains the detectors, and evaluates them; the trained model bundle is loaded by the controller at start-up.

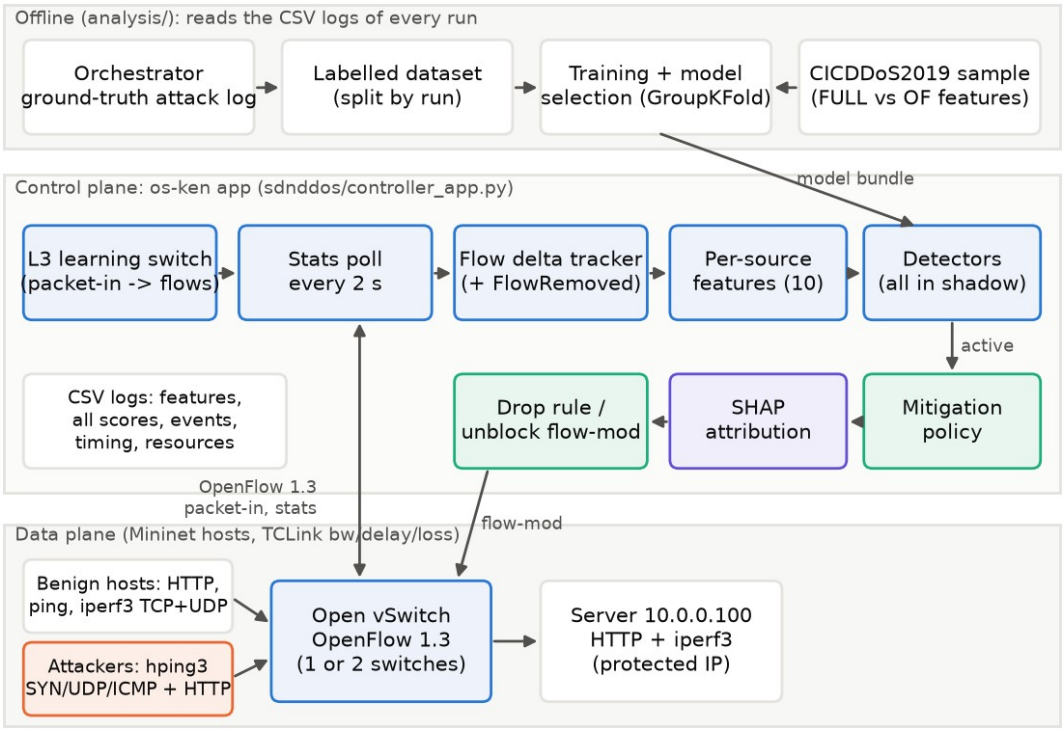



**Figure 1.** Architecture: data plane (bottom), os-ken controller application (middle) and offline training and evaluation pipeline (top).

## 3.1 Modules

**Table 2.** Modules of the framework and the files that implement them (brief items 1–11).

| Module | File(s) | Responsibility |
|---|---|---|
| 1. Topology + traffic | sdnddos/topo.py, traffic.py, benign_client.py | Parameterised topologies over TCLink; benign HTTP/ping/iperf3; hping3 SYN/UDP/ICMP floods confined to 10.0.0.0/24 |
| 2. Controller | sdnddos/controller_app.py, run_controller.py | OpenFlow 1.3 L3 learning switch; table-miss, forwarding and drop rules |
| 3. Flow statistics | controller_app.py, flowstats.py | Poll every 2 s; reassemble multipart replies; per-flow deltas with restart and FlowRemoved handling |
| 4. Feature extraction | sdnddos/features.py | Ten per-source features on the source's ingress switch |
| 5. Dataset preparation | analysis/build_dataset.py, labels.py, cic_prepare.py | Ground-truth labels from the orchestrator's attack log; CICDDoS2019 sampling; split by run |
| 6. ML detection | sdnddos/detectors.py, analysis/train.py | RF, XGBoost, autoencoder; GroupKFold model selection |

| Module | File(s) | Responsibility |
|---|---|---|
| 7. Baselines | sdnddos/detectors.py | No defense; tuned pps threshold; decision tree; logistic regression; RBF SVM |
| 8. Mitigation | sdnddos/mitigation.py | Confirmed blocking, quiet-based unblocking with back-off, protected server IP |
| 9. Explainability | sdnddos/explain.py, analysis/explain_offline.py | TreeSHAP per block decision (online) and globally (offline) |
| 10. Logging + monitoring | controller_app.py, orchestrator.py | Features and all detector scores per interval, events, timing, flows removed, CPU/RSS |
| 11. Evaluation + visualisation | analysis/eval_*.py, stats.py, plots.py | Metrics, bootstrap CIs, paired tests, figures |

### 3.2 Data flow in one polling interval

1. Every 2 s the controller sends a flow-statistics request to each switch.
2. For each reply, it keeps only forwarding rules (priority 10) and drop rules (priority 100) whose source IP entered the network at this switch, so a packet crossing two switches is counted once.
3. The flow-delta tracker converts cumulative counters into packets and bytes sent since the previous poll, adding the final counters of rules that expired in between.
4. Deltas are aggregated per source IP into the ten features of Section 4.1.
5. Every detector in the bundle scores every source (shadow mode); the scores are logged.
6. The active detector's verdict is fed to the mitigation policy, which may install or remove a drop rule. A block decision is logged with its top SHAP attributions.

## 4. Detection and mitigation design

### 4.1 Features

Forwarding rules match `(eth_type=IPv4, ipv4_src, ipv4_dst, ip_proto)`. Matching on IP addresses is what makes per-source attribution possible; the original prototype matched MAC addresses only and could not attribute traffic to a source IP (Section 9.3). Matching on transport ports was rejected, because a SYN flood with rotating source ports would then trigger a packet-in for every packet and turn the detector into an amplifier against its own controller. For each source $s$ and interval of length $\Delta t$ (measured, not assumed), the controller computes the features in Table 3.

**Table 3.** Per-source features computed by the controller every polling interval.

| Feature | Definition (over the interval) | Why |
|---|---|---|
| pps, bps | Packets and bits sent by s, forwarded or dropped, divided by Δt | Volume; drop-rule counters keep a blocked attacker visible |
| avg_pkt_size | Bytes / packets | Floods use uniform sizes (SYN ~60 B, UDP/ICMP 554 B) |
| n_flows | Active (s, dst, proto) rules with traffic | Breadth of activity |
| n_dst, n_proto | Distinct destinations and IP protocols | Benign hosts mix protocols; floods do not |
| new_flows | Rules first seen in the interval | Packet-in pressure on the controller |

| Feature | Definition (over the interval) | Why |
|---|---|---|
| frac_tcp, frac_udp, frac_icmp | Share of the source's packets per protocol | Protocol mix |

## 4.2 Detectors

All detectors implement one interface: a score per row and a threshold. Adding a detector is one new entry in a registry; neither the controller nor the evaluation code changes.

- **Static threshold (baseline b).** Flags a source whose pps exceeds a cut-off. The cut-off is *tuned* on the training runs to maximise F1 (171 pps), which makes it the strongest fair conventional baseline rather than a straw man.
- **Decision tree, logistic regression, RBF SVM (baseline c).** Depth-6 tree; logistic regression and SVM on signed-log-scaled, standardised features; balanced class weights; the SVM is Platt-calibrated.
- **Random forest and XGBoost (proposed candidates).** 200 trees with minimum leaf size 2, and 300 boosted trees of depth 4 (learning rate 0.1). Hyper-parameters were fixed a priori and not tuned on evaluation data.
- **Autoencoder (anomaly detector).** A 10–8–3–8–10 MLP trained only on benign intervals; an interval is flagged when its reconstruction error exceeds the 99th percentile of benign training error.

**Model selection rule (fixed before evaluation).** The proposed detector is whichever of random forest and XGBoost has the higher out-of-fold F1 in 5-fold cross-validation grouped by run on the training phase. XGBoost was selected (out-of-fold F1 0.996 vs 0.987 for random forest).

## 4.3 Mitigation policy

Figure 2 gives the block/unblock state machine. A source is blocked after the active detector flags it in **two consecutive** polling intervals in which the source sent traffic; the drop rule (priority 100, matching only `ipv4_src`) is installed on the source's ingress switch. Unblocking needs two conditions: the block has lasted at least 10 s, and the source has sent fewer than 20 pps for two consecutive polls. The rate is read from the drop rule's own counters. This detail matters: a blocked attacker looks silent on its forwarding rules precisely because it is being dropped, so an unblock rule based on forwarded traffic would release an active attacker immediately. A source that is blocked again has its minimum block time doubled. The server's address is protected from blocking for all detectors (a standard allow-list), so that a false positive on the server's reply traffic cannot turn the defense into a self-inflicted outage; its shadow detections are still reported.

```
ALLOWED --(flagged in 2 consecutive polls, source not protected)--> BLOCKED  [install drop
rule]
BLOCKED --(blocked >= min_block AND drop-rule pps < 20 for 2 polls)--> ALLOWED  [delete drop
rule]
min_block = 10 s x 2^(offences - 1)
```

**Figure 2.** Mitigation state machine (sdnddos/mitigation.py, covered by unit tests).

### 4.4 Explainability

For tree models the controller computes exact TreeSHAP values [18] for the blocked source's feature vector at the moment of blocking and stores the five largest contributions with the block event. For logistic regression the contribution is coefficient × standardised value; for the threshold it is the margin above the cut-off. Offline, the same explainer produces global importance and beeswarm plots over held-out data (Section 8.6).

## 5. Implementation

### 5.1 Stack and versions

**Table 4.** Software and hardware used for every experiment.

| Component | Version / detail |
|---|---|
| Host | Intel Core i7-13650HX (20 logical CPUs), 15 GiB RAM, CachyOS Linux, kernel 7.3.0, Docker 29.9.2 |
| Container | Ubuntu 24.04.5 LTS, privileged, --network none |
| Emulation | Mininet 2.3.0, Open vSwitch 3.3.9 (host kernel datapath), Linux HTB/netem via TCLink |
| Controller | os-ken 4.2.2 (OpenStack fork of Ryu), OpenFlow 1.3, Python 3.12.3 |
| Traffic | hping3 3.0.0a2, iperf3 3.16, ping (iputils), Python urllib HTTP client and http.server |
| ML | scikit-learn 1.9.1, XGBoost 3.4.1, SHAP 0.54.1, NumPy 2.5.4, pandas 3.1.0 |

Ryu 4.34 fails on current Python releases (eventlet incompatibility), so the controller uses os-ken, whose API is the same apart from the package name. os-ken 4.x ships no command-line manager; a 20-line launcher (sdnddos/run_controller.py) reproduces what ryu-manager did. os-ken 4.x runs its hub on native threads rather than eventlet greenlets. All of our event handlers run on the application's single event-loop thread, so controller state has a single writer; the polling thread only snapshots the switch list and enqueues requests.

### 5.2 Forwarding and statistics

The packet-in handler learns MAC-to-port mappings, records the first switch that reports each new source IP as that source's ingress switch, and installs an IP forwarding rule with a 10 s idle timeout and the send-flow-removed flag. Multipart statistics replies are reassembled before processing.

### 5.3 Counter handling

The flow-delta tracker is the piece that fixes the original prototype's silent under-counting. The listing below is its core; the unit tests cover new flows, normal deltas, a restart without a flow-removed message, a rule removed between polls, and a rule removed and re-created within one interval.

```
for s in samples:                                   # one poll of one switch
    prev = self._last.get(s.key)
    restarted = prev is not None and (s.packets < prev.packets or s.duration < prev.duration)
    if prev is None or restarted:
        dp, db = s.packets, s.bytes                     # new instance: all of it is new
    else:
        dp, db = s.packets - prev.packets, s.bytes - prev.bytes
    ...
```

```
for key, (dp, db, was_unseen) in self._pending.items():   # from OFPFlowRemoved
    ...add the final counters of expired instances to this interval
```

### 5.4 Shadow logging

In every evaluation run the controller loads all seven detectors and logs each one's score for each source and interval, together with the feature vector, whether the source is blocked, and per-detector inference time. Only the configured *active* detector influences the network. Offline, these logged scores were checked against scores recomputed from the logged features with the saved models; they agree to within floating-point rounding (assertion in analysis/eval_detection.py).

### 5.5 Orchestration and safety

One Python orchestrator runs one experiment end to end: it starts the controller, builds the topology, disables segmentation offload on every host interface (otherwise veth interfaces deliver 64 KB super-packets, a fault found in the original data), starts the servers, benign clients and probes, starts the attackers at the scheduled time with sub-second jitter, records each attacker's exact start and stop time on the controller's clock, and collects all logs. Attack commands are built only for targets inside 10.0.0.0/24, and the container has no network interface other than loopback, so emulated attack traffic cannot leave it.

## 6. Datasets

### 6.1 CICDDoS2019

The two CSV archives (day 01-12, 2.33 GB; day 03-11, 0.92 GB, compressed) were obtained from a public mirror of the original files because the official download requires a registration form; their SHA-256 hashes are recorded in the repository so that they can be checked against an official copy. The archives were streamed without extraction. Every BENIGN row was kept, and attack rows were Bernoulli-sampled to about 40 000 per CSV file (seed 2019), giving 889,321 rows. Table 5 lists the result. Identifier columns (flow ID, IP addresses, ports, timestamp, the Inbound flag and the SimillarHTTP field) were dropped before training because they identify capture sessions rather than behaviour.

**Table 5.** CICDDoS2019 stratified sample: rows per day and class.

| Day | Benign rows (all kept) | Attack rows (sampled) | Attack types |
|---|---|---|---|
| 01-12 | 56,863 | 469,204 | DrDoS_DNS, DrDoS_LDAP, DrDoS_MSSQL, DrDoS_NTP, DrDoS_NetBIOS, DrDoS_SNMP, DrDoS_SSDP, DrDoS_UDP, Syn, TFTP, UDP-lag, WebDDoS |
| 03-11 | 56,965 | 306,289 | LDAP, MSSQL, NetBIOS, Portmap, Syn, UDP, UDPLag |

Two feature sets are compared. **FULL** uses all remaining numeric CICFlowMeter features (78). **OF** uses only what an OpenFlow 1.3 switch reports for a unidirectional flow: duration, forward packets and forward bytes (CICFlowMeter's payload length plus a per-packet header estimate

of 54 B for TCP and 42 B otherwise, because OpenFlow counts frame bytes), protocol, and the rates and mean size derived from them.

### 6.2 Mininet traces

Mininet data come from the controller's own logs. Each row is one (source, polling interval) with the ten features of Table 3. Labels are computed from the orchestrator's attack log: a row is *attack* if the source is an attacker and the interval overlaps that attacker's attack window; all other rows, including the attackers' own browsing outside the window, are *benign*. Attackers browse before, during and after the attack, so host identity cannot serve as a shortcut, and the protected server's reply traffic is labelled benign.

**Splits by run.** The training phase (A) and the evaluation phases (B, C, O) are disjoint sets of runs with disjoint seeds, generated by different plans. Within phase A, model selection uses GroupKFold with the run as the group, so intervals of one run never appear on both sides of a split. No row of phases B or C was used for training, tuning or model selection.

**Table 6.** Mininet datasets (rows = source x polling interval).

| Phase | Runs | Rows | Attack rows | Use |
|---|---|---|---|---|
| A | 72 | 15,505 | 3,212 | Training and model selection (GroupKFold by run) |
| B | 180 | 34,037 | 8,632 | Held-out evaluation, core matrix |
| C | 195 | 44,585 | 9,345 | Held-out evaluation, network conditions |

## 7. Experimental setup

### 7.1 Topologies and traffic

The **small** topology has one switch connecting the server (10.0.0.100), three benign hosts and three attackers. The **large** topology has two switches, eight benign hosts and four attackers, with hosts alternating between switches, so attack traffic from the second switch crosses the trunk. Host links have bandwidth $B$, delay $d$ and loss $l$; the trunk has $4B$, so the server's link is the bottleneck. Benign host b1 pings the server every 0.2 s (latency probe), b2 runs an iperf3 TCP stream at $0.4B$ (throughput probe), b3 runs an iperf3 UDP stream at $0.1B$, and every benign host fetches 1 kB, 10 kB or 100 kB files over HTTP with exponentially distributed think times (mean 2 s, 0.5 s or 0.1 s for low, medium and high benign load). Attackers browse with a 2 s mean think time and, during the attack window, each runs hping3 at the configured rate (Figure 3).

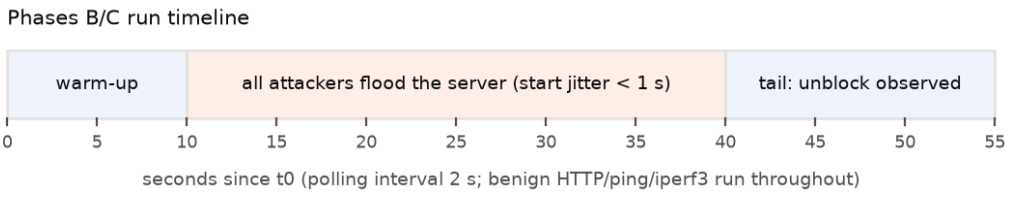



**Figure 3.** Timeline of every evaluation run. Labels, baselines and attack-window metrics use these windows, recorded on the controller's clock.

## 7.2 Experiment matrix

**Table 7.** Experiment phases. Every evaluation condition is repeated 5 times; a repetition uses the same seed for every defense mode.

| Phase | Conditions | Defense modes | Reps | Runs |
|---|---|---|---|---|
| A (train) | 60 attack runs: random type, log-uniform rate 30–8 000 pps, random load, link and topology; 12 benign-only runs | none (no models yet) | 1 | 72 |
| B (core) | 3 attack types x 3 rates (50/500/5 000 pps per attacker); small topology; 10 Mb/s, 2 ms, 0 % loss; medium load | none, threshold, DT, XGBoost | 5 | 180 |
| C (conditions) | Medium rate x 3 types, one factor changed at a time: high benign load; 50 Mb/s links; WAN (20 ms, 1 % loss); large topology. Plus a benign-only flash crowd (high load) | none, threshold, XGBoost | 5 | 195 |

Phases A, B and C ran on four (A: three) containers in parallel; each container has its own network namespace and OVS instance, and run order was shuffled with a fixed seed so that any interference is spread evenly across modes. A full factorial over all factors (3 x 3 x 3 x 3 x 2 conditions x 5 reps x 4 modes) would need over 3 000 runs; the one-factor-at-a-time design of phase C is a deliberate compromise and is listed as a limitation.

## 7.3 Metrics

- **Detection** (per source x interval, phases B and C, runs with no active defense so that no detector changes the traffic): accuracy, precision, recall, F1, false-positive rate (FPR), ROC-AUC, PR-AUC; 95 % confidence intervals by bootstrap over runs (1 000 resamples).
- **Detection latency:** time from an attacker's start to the end of its first flagged interval; **simulated block latency** uses the two-interval confirmation.
- **Time to mitigation (TTM):** time from an attacker's start to the installation of its drop rule (closed-loop runs).
- **Service quality during the attack window:** benign TCP throughput relative to the same run's pre-attack baseline; benign UDP loss; ping RTT median and 95th percentile and ping loss; HTTP success rate and median response time of benign hosts.
- **Collateral damage:** false blocks (blocks of non-attackers) and seconds that benign sources spent blocked.
- **Overhead:** per-detector inference time per poll (measured live in shadow mode), total poll processing time, controller CPU and memory.

**Statistics.** Per condition we report mean, standard deviation and a Student-t 95 % CI over 5 repetitions. Comparisons between modes are paired on (condition, repetition), which share a seed. With n = 5 pairs a two-sided Wilcoxon signed-rank test cannot reach $p < 0.0625$, so per-condition differences are reported as bootstrap 95 % CIs of the mean paired difference, and significance tests (Wilcoxon) are pooled over all pairs of a phase.

# 8. Results

All numbers in this section come from the tables in results/tables/ of the repository; table and figure captions name the experiment phase. Unless stated otherwise, ± values are half-widths of 95 % Student-t confidence intervals over 5 seed-paired repetitions, and bracketed ranges are 95 % bootstrap intervals.

## 8.1 Attack generation sanity check

Table 8 compares the configured attack rate with what hping3 actually sent (its own counter) and with what the controller measured for the attacker at the ingress switch (median over attack intervals, undefended phase-B runs). Low and medium rates are met almost exactly. At the high rate hping3 sends about 95 % of the nominal 5 000 pps. The controller sees two systematic differences, which matter when reading every later table. **SYN floods appear at about twice the sent rate**, because each SYN-ACK from the server is answered by an RST from the attacker's own kernel, and the RST shares the (source, destination, TCP) flow key. **UDP and ICMP floods at the high level are capped by the attacker's 10 Mb/s access link** (554-byte frames, about 2 260 pps), so 'high' means *link-saturating* for them. Tables keep the nominal labels.

**Table 8.** Attack rate per attacker: configured, sent by hping3, and measured by the controller at the ingress switch (phase B, mode none).

| Condition | Nominal pps | Sent (hping3, mean) | Seen at switch (median) | Attackers |
|---|---|---|---|---|
| SYN low | 50 | 50 | 102 | 15 |
| SYN medium | 500 | 491 | 993 | 15 |
| SYN high | 5000 | 4583 | 8509 | 15 |
| UDP low | 50 | 50 | 53 | 15 |
| UDP medium | 500 | 481 | 494 | 15 |
| UDP high | 5000 | 4465 | 2259 | 15 |
| ICMP low | 50 | 50 | 53 | 15 |
| ICMP medium | 500 | 479 | 486 | 15 |
| ICMP high | 5000 | 4578 | 2258 | 15 |

## 8.2 Training and model selection

Table 9 lists out-of-fold results on phase A (15,558 intervals, 3,213 of them attacks), with 5-fold cross-validation grouped by run. XGBoost and random forest were the two candidates for the proposed detector; XGBoost had the higher F1 and was selected. The RBF SVM came close, and the decision tree was within two points of F1. The autoencoder flagged no attack interval at its a-priori threshold (99th percentile of benign error): its highest errors came from the server's own reply traffic, whose many simultaneous flows are rare in the benign data, while single-protocol floods are easy to reconstruct. Its ROC-AUC of 0.858 shows that the score carries some signal, but with an unsupervised threshold it is not usable as a detector here. We did not re-tune its threshold on attack labels, because that would turn it into a supervised model.

**Table 9.** Phase A: out-of-fold detection results (GroupKFold by run, 5 folds) for the deployed models, and F1 after re-training on the cleaned phase-A data (see text).

| Detector | F1 | ROC-AUC | F1, cleaned A | Selection |
|---|---|---|---|---|
| XGBoost | 0.996 | 0.999 | 0.997 | proposed |
| RBF SVM | 0.992 | 0.999 | 0.993 | |
| Random forest | 0.987 | 0.999 | 0.997 | |
| Decision tree | 0.980 | 0.983 | 0.992 | |
| Logistic regression | 0.804 | 0.979 | 0.800 | |
| Static threshold | 0.527 | 0.803 | 0.518 | |
| Autoencoder | 0.000 | 0.858 | 0.002 | |

**A data defect found after training.** An independent code review during the evaluation found that, in the two-switch topology, flow-removed messages from the *transit* switch bypassed the ingress filter and created 53 spurious feature rows in phase A (lifetime counters reported as one interval). The controller was fixed before phase C, and the analysis drops such rows (phase B uses one switch and is unaffected). The deployed models had already been trained on the uncleaned data and were used unchanged in all evaluation runs. Re-training on the cleaned data selects the same model (XGBoost) and the same threshold (171 pps), and raises out-of-fold F1 slightly for most models (last column of Table 9), so the evaluation slightly *under*-states what the trained models can do.

## 8.3 Detection on held-out runs

Table 10 and Figure 4 report detection on the undefended runs of phases B and C (23,220 source-intervals, 5,271 of them attacks), scored live by every detector in shadow mode. None of these runs was used for training or model selection, and their seeds differ from phase A. Re-scoring the logged features offline reproduced every live decision (at most 0 mismatches per detector over 23,220 rows; Appendix C).

**Table 10.** Held-out detection (phases B + C, mode none). F1/recall/FPR with 95 % bootstrap CIs over runs. Latency = seconds from attack start to the first flagged interval; block latency adds the two-interval confirmation; missed = attackers never flagged twice in a row during their attack; n/a = never flagged.

| Detector | F1 [CI] | Prec. | Recall [CI] | FPR % [CI] | AUC | Lat. (s) | Block lat. (s) | Missed % |
|---|---|---|---|---|---|---|---|---|
| Random forest | 0.998 [0.997, 0.999] | 0.999 | 0.998 [0.996, 0.999] | 0.03 [0.00, 0.07] | 1.000 | 1.18 | 3.18 | 0.0 |
| XGBoost | 0.998 [0.996, 0.999] | 0.999 | 0.996 [0.994, 0.998] | 0.03 [0.01, 0.07] | 1.000 | 1.23 | 3.23 | 0.0 |
| RBF SVM | 0.996 [0.995, 0.998] | 0.999 | 0.994 [0.991, 0.996] | 0.02 [0.00, 0.06] | 0.999 | 1.26 | 3.26 | 0.0 |
| Decision tree | 0.994 [0.992, 0.996] | 0.992 | 0.995 [0.992, 0.997] | 0.22 [0.14, 0.32] | 0.996 | 1.24 | 3.24 | 0.0 |
| Logistic regression | 0.919 [0.894, 0.940] | 0.910 | 0.928 [0.888, 0.960] | 2.69 [1.66, 3.94] | 0.992 | 1.78 | 3.96 | 2.1 |
| Static threshold | 0.570 [0.527, 0.605] | 0.431 | 0.840 [0.774, 0.899] | 32.51 [29.68, 35.86] | 0.822 | 1.71 | 3.61 | 13.6 |
| Autoencoder | 0.000 [0.000, 0.000] | 0.000 | 0.000 [0.000, 0.000] | 0.91 [0.83, 0.98] | 0.860 | n/a | n/a | 100.0 |

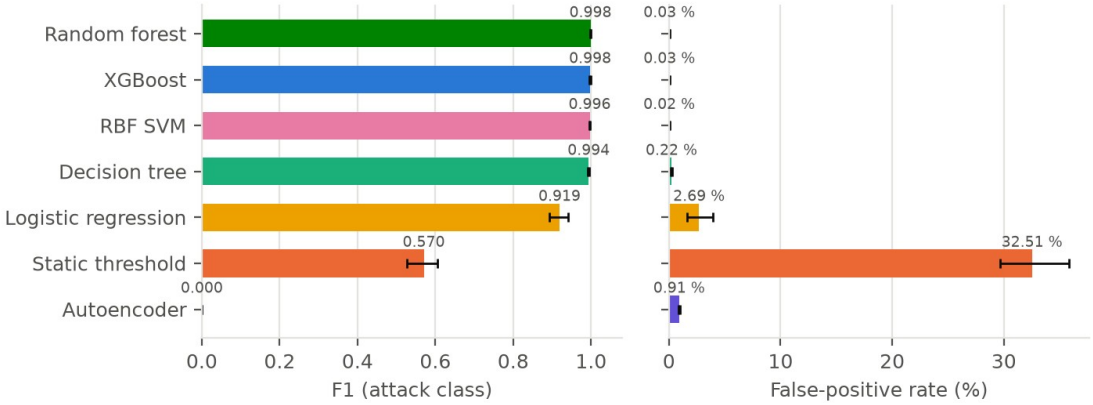



**Figure 4.** Held-out detection F1 and false-positive rate per detector (phases B + C, mode none), with 95 % bootstrap CIs over runs.

Three results stand out. First, every supervised model clearly beats the tuned threshold: XGBoost reached F1 0.998 against 0.570. The threshold flagged 32.51 % of benign intervals: 26.3 % of benign-host intervals (mostly the iperf3 host, which sends about 350 pps of full-size packets, above the 171 pps cut-off) and 85.1 % of the server's intervals (its replies). Second, the gap between XGBoost and the other trained models is small. The decision tree reached 0.994, the random forest 0.998 and the SVM 0.996. Table 11 tests the per-run differences: XGBoost's F1 advantage over the decision tree is +0.003 [+0.002, +0.005] (Wilcoxon p = < 0.001), statistically clear but tiny, and the random forest is slightly *better* than XGBoost (-0.001, p = 0.011). The pre-registered selection rule picked XGBoost on training data; on held-out data the two ensembles are practically equivalent. Third, detection is fast: all trained models flag an attacker 1.23 s after it starts on average, which is one polling interval plus the offset of the attack start inside it. With two-interval confirmation the expected block latency is 3.23 s.

**Table 11.** Paired per-run comparison of XGBoost with each baseline (phases B + C, mode none). Difference = XGBoost − baseline; positive is better for F1, negative is better for FPR.

| Baseline | Metric | Runs | Mean diff. | 95 % CI | Wilcoxon p |
|---|---|---|---|---|---|
| Autoencoder | F1 | 110 | +0.9521 | [+0.9076, +0.9883] | < 0.001 |
| Autoencoder | FPR | 110 | -0.0091 | [-0.0099, -0.0083] | < 0.001 |
| Decision tree | F1 | 110 | +0.0032 | [+0.0018, +0.0048] | < 0.001 |
| Decision tree | FPR | 110 | -0.0021 | [-0.0031, -0.0013] | < 0.001 |
| Logistic regression | F1 | 110 | +0.0852 | [+0.0579, +0.1152] | < 0.001 |
| Logistic regression | FPR | 110 | -0.0306 | [-0.0439, -0.0189] | < 0.001 |
| Random forest | F1 | 110 | -0.0008 | [-0.0015, -0.0003] | 0.011 |
| Random forest | FPR | 110 | +0.0001 | [+0.0000, +0.0002] | 0.317 |
| RBF SVM | F1 | 110 | +0.0012 | [+0.0002, +0.0023] | 0.017 |
| RBF SVM | FPR | 110 | +0.0001 | [-0.0001, +0.0004] | 0.144 |
| Static threshold | F1 | 110 | +0.4082 | [+0.3609, +0.4599] | < 0.001 |
| Static threshold | FPR | 110 | -0.3502 | [-0.3804, -0.3192] | < 0.001 |

## 8.4 Detection by attack type and rate

Table 12 breaks recall down by condition, and Figure 5 plots it. The threshold detects high-rate floods but misses most low-rate ones, which send 50 pps, far below the benign iperf3 host. The trained models detect low-rate floods as well, because they rely mainly on packet size and

protocol mix rather than on volume (Section 8.7). Figure 6 shows that the threshold's weakness is structural, not a tuning problem: sweeping the cut-off over the held-out data, the best achievable F1 is 0.654 (at 449 pps, recall 0.761, FPR 16.6 %). No single pps value separates a 50 pps flood from a 350 pps benign transfer.

**Table 12.** Recall and FPR by attack type and rate (phase B, mode none, 5 runs each).

| Condition | Recall XGB | RF | SVM | DT | Thr. | FPR % XGB | DT | Thr. |
|---|---|---|---|---|---|---|---|---|
| SYN low (50 pps) | 0.979 | 0.996 | 0.954 | 0.988 | 0.004 | 0.00 | 0.14 | 39.57 |
| SYN medium (500 pps) | 1.000 | 1.000 | 1.000 | 1.000 | 0.996 | 0.00 | 0.00 | 40.55 |
| SYN high (5000 pps) | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 0.14 | 1.28 | 31.38 |
| UDP low (50 pps) | 0.992 | 0.992 | 0.983 | 0.988 | 0.000 | 0.28 | 0.28 | 39.75 |
| UDP medium (500 pps) | 0.996 | 0.996 | 0.996 | 0.996 | 0.958 | 0.00 | 0.14 | 35.56 |
| UDP high (5000 pps) | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 0.00 | 0.14 | 14.91 |
| ICMP low (50 pps) | 0.988 | 0.992 | 0.971 | 0.958 | 0.000 | 0.00 | 0.00 | 40.61 |
| ICMP medium (500 pps) | 0.996 | 0.996 | 0.996 | 0.996 | 0.958 | 0.00 | 0.00 | 38.81 |
| ICMP high (5000 pps) | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 0.00 | 0.00 | 26.01 |

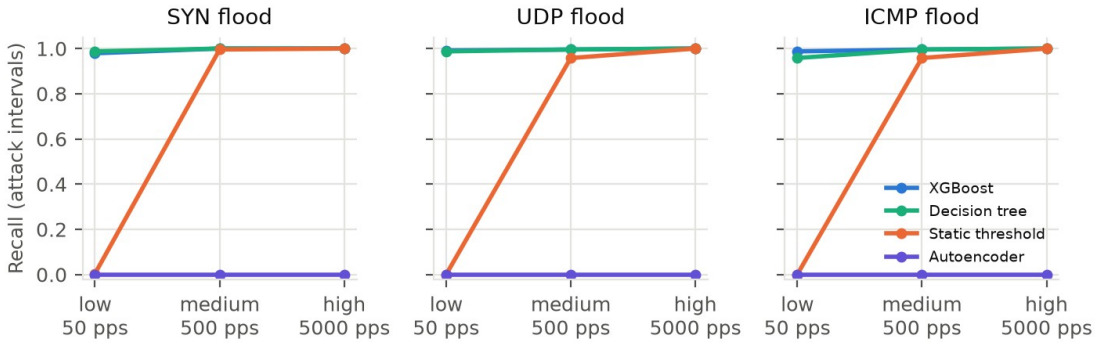



**Figure 5.** Recall by attack type and rate for XGBoost, decision tree, threshold and autoencoder (phase B, mode none).

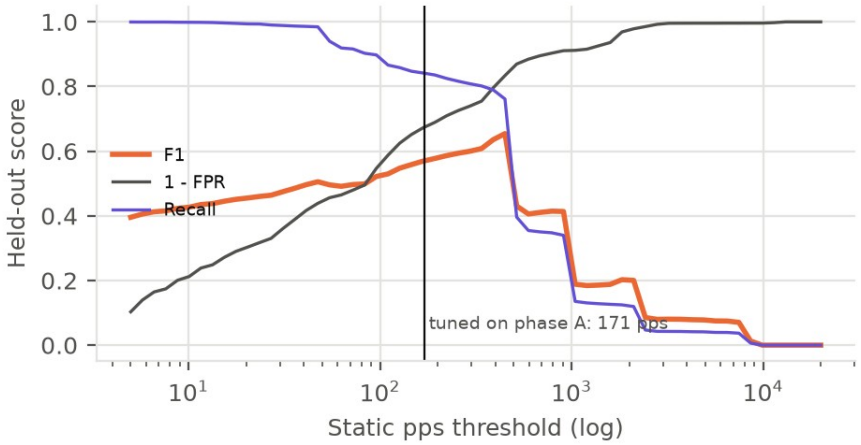



**Figure 6.** Static threshold sensitivity on held-out data: F1, recall and 1 − FPR as the pps cut-off varies. The vertical line marks the value tuned on phase A.

## 8.5 Closed-loop mitigation: core matrix (phase B)

In the closed-loop runs the active detector actually installs drop rules. Tables Table 13–Table 16 and Figure 8 compare four modes on the nine core conditions; each cell is the mean over five seed-paired runs. Figure 7 shows one run per mode for the high-rate UDP flood.

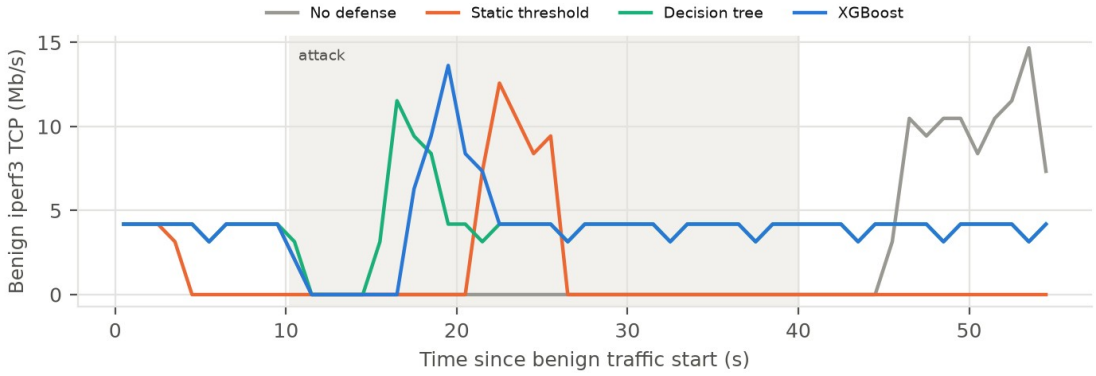



**Figure 7.** Benign iperf3 TCP throughput over time in one UDP 5 000 pps run (repetition 0), same seed for all modes. Shaded: attack window. After a stall, iperf3's rate limiter sends its backlog, so short peaks exceed the 4 Mb/s offered rate. Under the threshold the probe host itself is blocked from second 4, apart from one unblock-and-reblock cycle around seconds 21–26.

**Table 13.** Benign TCP goodput during the attack as a fraction of the offered rate (phase B). Mean ± 95 % CI half-width, n = 5.

| Condition | No defense | Threshold | Decision tree | XGBoost |
|---|---|---|---|---|
| SYN low | 1.00 ±0.00 | 0.36 ±0.02 | 1.00 ±0.00 | 1.00 ±0.00 |
| SYN medium | 1.00 ±0.00 | 0.38 ±0.03 | 1.00 ±0.00 | 1.00 ±0.00 |
| SYN high | 0.10 ±0.06 | 0.36 ±0.06 | 1.00 ±0.00 | 1.00 ±0.00 |
| UDP low | 1.00 ±0.00 | 0.31 ±0.07 | 1.00 ±0.00 | 1.00 ±0.00 |
| UDP medium | 0.71 ±0.09 | 0.36 ±0.09 | 1.00 ±0.00 | 1.00 ±0.00 |
| UDP high | 0.04 ±0.03 | 0.38 ±0.04 | 1.00 ±0.00 | 1.00 ±0.00 |
| ICMP low | 1.00 ±0.00 | 0.36 ±0.07 | 1.00 ±0.00 | 1.00 ±0.00 |
| ICMP medium | 0.73 ±0.08 | 0.34 ±0.05 | 1.00 ±0.00 | 1.00 ±0.00 |
| ICMP high | 0.03 ±0.03 | 0.37 ±0.06 | 1.00 ±0.00 | 1.00 ±0.00 |

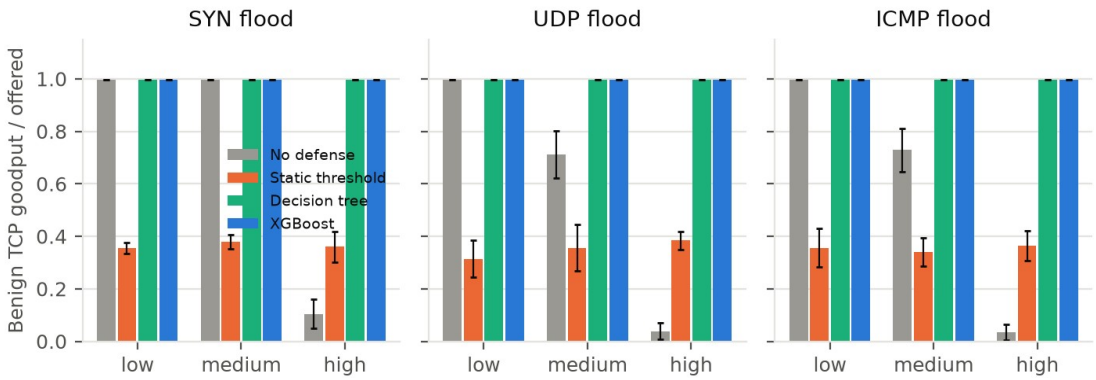



**Figure 8.** Benign TCP goodput / offered rate during the attack, per mode and condition (phase B, mean and 95 % CI).

Without a defense, low-rate floods do no measurable harm to the TCP probe. Medium-rate UDP and ICMP floods cut its goodput to about three quarters of the offered rate, and high-rate floods nearly stop it. SYN floods only hurt at the high rate, because their 60-byte packets fill the link far more slowly. With XGBoost or the decision tree, goodput stays at the offered rate in every condition. The threshold behaves very differently. It blocks the benign iperf3 host itself, usually during the warm-up before any attack starts, and blocks it again after every unblock, so the TCP probe gets about a third of its offered rate in *every* condition, including those where the attack alone would have done no harm. Averaged over all nine conditions, goodput was 1.00 with XGBoost, 1.00 with the decision tree, 0.36 with the threshold and 0.62 with no defense.

**Table 14.** Median ping RTT of the benign probe during the attack, in ms (phase B). Mean ± 95 % CI half-width, n = 5.

| Condition | No defense | Threshold | Decision tree | XGBoost |
|---|---|---|---|---|
| SYN low | 11.1 ±0.9 | 8.1 ±0.0 | 10.0 ±1.2 | 10.3 ±1.6 |
| SYN medium | 15.1 ±0.8 | 8.1 ±0.0 | 10.5 ±0.7 | 10.1 ±0.7 |
| SYN high | 54.5 ±1.4 | 8.1 ±0.1 | 12.4 ±0.5 | 13.1 ±0.9 |
| UDP low | 12.5 ±0.4 | 8.1 ±0.0 | 10.5 ±0.8 | 9.4 ±1.3 |
| UDP medium | 510.1 ±32.7 | 8.1 ±0.1 | 12.3 ±0.7 | 12.7 ±0.9 |
| UDP high | 456.3 ±0.6 | 8.1 ±0.1 | 13.3 ±0.9 | 13.8 ±2.6 |
| ICMP low | 12.8 ±1.6 | 8.1 ±0.1 | 9.9 ±1.1 | 10.8 ±1.3 |
| ICMP medium | 486.4 ±59.4 | 8.1 ±0.0 | 12.5 ±1.2 | 12.7 ±1.7 |
| ICMP high | 456.7 ±1.0 | 8.1 ±0.1 | 14.2 ±1.3 | 14.2 ±1.3 |

Latency tells the same story (Table 14; plotted in Figure 13, Appendix C). An undefended flood that fills the bottleneck queue raises the median RTT from about 8 ms to hundreds of milliseconds. Under XGBoost and the decision tree, RTT returns to near its idle value once the attackers are blocked. In every condition the threshold shows the *lowest* RTT, a few milliseconds below XGBoost. The reason is not better mitigation: the threshold has blocked the benign iperf3 stream, which empties the queue the ping probe shares. The same false blocks lower the threshold's HTTP success rate (Table 15), because the blocked iperf3 host also browses. Section 8.9 lists these cases.

**Table 15.** Benign HTTP success rate during the attack, % of requests (phase B). Mean ± 95 % CI half-width, n = 5.

| Condition | No defense | Threshold | Decision tree | XGBoost |
|---|---|---|---|---|
| SYN low | 100.0 ±0.0 | 97.2 ±0.6 | 100.0 ±0.0 | 100.0 ±0.0 |
| SYN medium | 100.0 ±0.0 | 97.5 ±0.5 | 100.0 ±0.0 | 100.0 ±0.0 |
| SYN high | 100.0 ±0.0 | 97.5 ±0.4 | 100.0 ±0.0 | 100.0 ±0.0 |
| UDP low | 100.0 ±0.0 | 97.4 ±0.2 | 100.0 ±0.0 | 100.0 ±0.0 |
| UDP medium | 100.0 ±0.0 | 97.3 ±0.5 | 100.0 ±0.0 | 100.0 ±0.0 |
| UDP high | 42.8 ±18.3 | 96.9 ±0.8 | 99.0 ±0.9 | 99.6 ±1.0 |
| ICMP low | 100.0 ±0.0 | 97.5 ±1.1 | 100.0 ±0.0 | 100.0 ±0.0 |
| ICMP medium | 100.0 ±0.0 | 97.8 ±0.3 | 100.0 ±0.0 | 100.0 ±0.0 |
| ICMP high | 35.6 ±11.8 | 96.9 ±0.8 | 99.8 ±0.5 | 99.8 ±0.4 |

**Table 16.** Time to mitigation (s, mean ± CI half-width) and false blocks per run (mean) in phase B. Time to mitigation is measured per attacker from its own start; n/a = no attacker blocked during its attack. Per-run values are plotted in Figure 14 (Appendix C).

| Condition | TTM thr. | TTM DT | TTM XGB | FB thr. | FB DT | FB XGB |
|---|---|---|---|---|---|---|
| SYN low | n/a | 3.65 ±0.40 | 3.58 ±0.86 | 2.0 | 0.0 | 0.0 |
| SYN medium | 3.28 ±0.10 | 3.49 ±0.59 | 2.99 ±0.50 | 2.4 | 0.0 | 0.0 |
| SYN high | 3.33 ±0.32 | 3.12 ±0.42 | 3.24 ±0.12 | 2.2 | 0.0 | 0.0 |
| UDP low | n/a | 3.19 ±0.33 | 3.09 ±0.24 | 2.8 | 0.0 | 0.0 |
| UDP medium | 3.96 ±0.77 | 3.29 ±0.16 | 3.01 ±0.42 | 2.6 | 0.0 | 0.0 |
| UDP high | 2.93 ±0.40 | 2.95 ±0.49 | 3.21 ±0.59 | 2.6 | 0.0 | 0.0 |
| ICMP low | n/a | 3.54 ±0.59 | 3.29 ±0.36 | 2.4 | 0.0 | 0.0 |
| ICMP medium | 3.56 ±0.50 | 3.45 ±0.16 | 3.19 ±0.53 | 2.2 | 0.0 | 0.0 |
| ICMP high | 3.23 ±0.31 | 3.27 ±0.31 | 3.11 ±0.35 | 2.2 | 0.0 | 0.0 |

Trained detectors blocked every attacker in every core condition, on average 3.19 s (XGBoost) and 3.33 s (decision tree) after it started, which matches the simulated two-interval latency of Section 8.3. The threshold blocked only 67 % of attackers during their attacks, missing the low-rate ones, and caused 2.38 false blocks per run on average, against 0.00 for XGBoost.

## 8.6 Network conditions (phase C)

Phase C changes one factor at a time at the medium attack rate. Table 17 and Figure 9 summarise goodput and RTT. XGBoost keeps goodput at the offered rate under high benign load, 50 Mb/s links and the two-switch topology, and it is never worse than no defense. The conditions themselves matter. In the large topology four attackers flood instead of three, so the undefended network suffers more than in phase B. With 50 Mb/s links the medium-rate attack no longer saturates the bottleneck, so even the undefended network keeps its goodput and the defense has nothing to gain. On the WAN profile, 1 % random loss limits TCP to about a third of the offered rate for *every* mode, so no defense can restore it there, and RTT is dominated by the 80 ms path delay. The threshold again blocks the iperf3 host in every condition.

**Table 17.** Phase C: benign TCP goodput / offered rate and median ping RTT (ms) at 500 pps per attacker. Mean ± 95 % CI half-width, n = 5.

| Condition | Goodput none | Thr. | XGB | RTT none | Thr. | XGB |
|---|---|---|---|---|---|---|
| SYN, high benign load | 1.00 ±0.00 | 0.34 ±0.15 | 1.00 ±0.00 | 22.2 ±1.4 | 8.1 ±0.0 | 15.6 ±1.1 |
| SYN, 50 Mb/s links | 1.00 ±0.00 | 0.31 ±0.06 | 1.00 ±0.00 | 8.1 ±0.1 | 8.1 ±0.0 | 8.1 ±0.0 |
| SYN, WAN: 20 ms, 1 % loss | 0.26 ±0.04 | 0.22 ±0.08 | 0.32 ±0.02 | 80.0 ±0.1 | 80.0 ±0.1 | 80.0 ±0.1 |
| SYN, large topology | 1.00 ±0.00 | 0.36 ±0.04 | 1.00 ±0.00 | 22.9 ±3.8 | 8.1 ±0.0 | 12.8 ±1.7 |
| UDP, high benign load | 0.70 ±0.06 | 0.38 ±0.08 | 1.00 ±0.00 | 473.9 ±23.0 | 8.1 ±0.0 | 17.1 ±1.3 |
| UDP, 50 Mb/s links | 1.00 ±0.00 | 0.29 ±0.03 | 1.00 ±0.00 | 9.0 ±0.2 | 8.0 ±0.0 | 8.1 ±0.0 |
| UDP, WAN: 20 ms, 1 % loss | 0.27 ±0.02 | 0.26 ±0.09 | 0.28 ±0.02 | 80.1 ±0.1 | 80.0 ±0.1 | 80.0 ±0.1 |
| UDP, large topology | 0.19 ±0.02 | 0.37 ±0.06 | 1.00 ±0.00 | 456.8 ±4.4 | 8.1 ±0.1 | 17.1 ±2.7 |
| ICMP, high benign load | 0.67 ±0.03 | 0.38 ±0.04 | 1.00 ±0.00 | 477.0 ±33.1 | 8.0 ±0.0 | 18.1 ±2.3 |
| ICMP, 50 Mb/s links | 1.00 ±0.00 | 0.32 ±0.04 | 1.00 ±0.00 | 9.0 ±0.1 | 8.1 ±0.0 | 8.1 ±0.1 |
| ICMP, WAN: 20 ms, 1 % loss | 0.30 ±0.04 | 0.24 ±0.10 | 0.32 ±0.04 | 80.5 ±0.7 | 80.1 ±0.1 | 80.0 ±0.0 |
| ICMP, large topology | 0.27 ±0.11 | 0.36 ±0.06 | 1.00 ±0.00 | 482.1 ±24.3 | 8.1 ±0.1 | 17.4 ±2.4 |

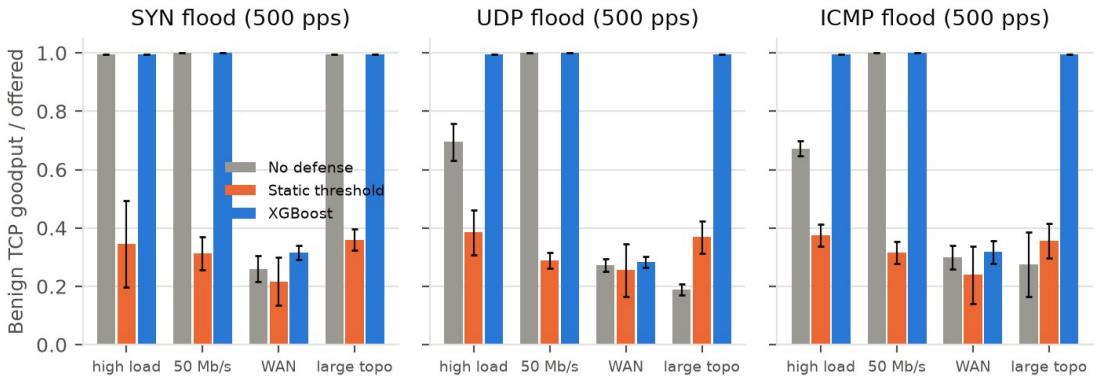



**Figure 9.** Phase C: benign TCP goodput / offered rate under changed network conditions.

The flash-crowd condition has no attacker at all, only high benign load, so every block is a false block (Table 18). XGBoost blocked 0.0 benign sources per run on average, against 4.4 for the threshold.

**Table 18.** Flash crowd (benign only, high load; phase C, n = 5): false blocks, seconds benign sources spent blocked, TCP goodput and HTTP success.

| Mode | False blocks / run | Benign blocked (s) | TCP goodput / offered | HTTP success % |
|---|---|---|---|---|
| No defense | 0.0 | 0.0 | 1.00 ±0.00 | 100.0 ±0.0 |
| Static threshold | 4.4 | 95.9 | 0.33 ±0.13 | 96.3 ±0.9 |
| XGBoost | 0.0 | 0.0 | 1.00 ±0.00 | 100.0 ±0.0 |

## 8.7 Explainability

Figure 10 shows global TreeSHAP attributions of XGBoost on held-out phase-B intervals, and Table 19 lists mean |SHAP| for the three tree models. The most influential features are `avg_pkt_size`, `frac_tcp`, `bps`, `n_dst`; the packet rate (pps) contributes little. This explains both why the trained models catch low-rate floods and why they do not flag the benign iperf3 host: what separates the classes in our traffic is mainly packet-size and protocol homogeneity, not packet rate. The same dependency is a risk, because an attacker who mimics benign packet sizes and mixes protocols would be harder to detect (Section 9.2).

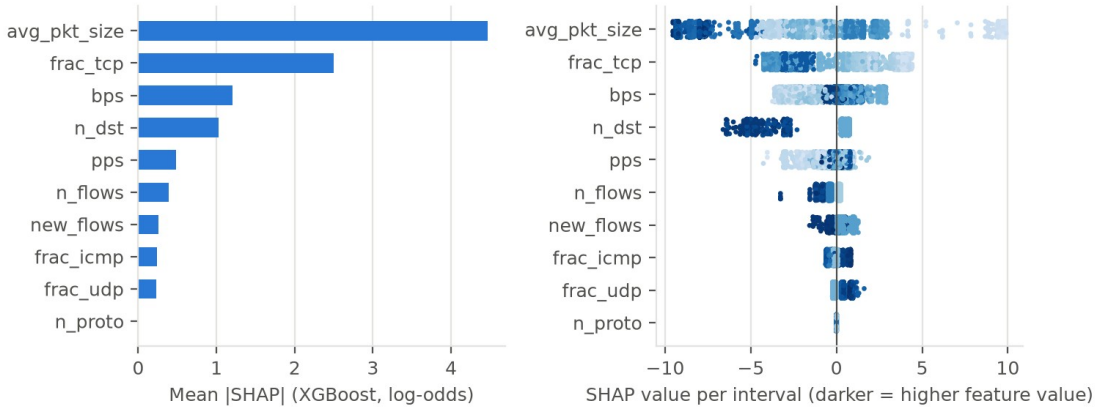



**Figure 10.** XGBoost SHAP attributions on 3 000 held-out phase-B intervals: mean |SHAP| (left) and per-interval values coloured by feature value (right).

**Table 19.** Mean |SHAP| per feature (log-odds for XGBoost; probability for random forest and decision tree), held-out phase B.

| Feature | XGBoost | Random forest | Decision tree |
|---|---|---|---|
| avg_pkt_size | 4.465 | 0.217 | 0.267 |
| frac_tcp | 2.497 | 0.076 | 0.009 |
| bps | 1.201 | 0.077 | 0.251 |
| n_dst | 1.025 | 0.036 | 0.109 |
| pps | 0.484 | 0.067 | 0.001 |
| n_flows | 0.388 | 0.028 | 0.000 |
| new_flows | 0.258 | 0.001 | 0.004 |
| frac_icmp | 0.241 | 0.037 | 0.000 |
| frac_udp | 0.233 | 0.024 | 0.030 |
| n_proto | 0.005 | 0.005 | 0.000 |

Per-decision explanations were logged online for all 330 XGBoost block decisions in phases B and C. Table 20 counts which feature had the largest attribution in each. The top feature was `frac_tcp` in 64.8 % of the decisions. Each block event in events.csv carries its five largest attributions, so an operator can see why a source was blocked without re-running the model.

**Table 20.** Top-ranked SHAP feature of every XGBoost block decision (phases B and C).

| Top feature | Block decisions | Share |
|---|---|---|
| frac_tcp | 214 | 64.8 % |
| avg_pkt_size | 112 | 33.9 % |
| bps | 4 | 1.2 % |

## 8.8 Overhead

Because every detector ran in shadow mode in every evaluation run, the controller timed each detector on every poll. Table 21 averages these live timings over 12,520 polls with traffic (on average 6.3 sources per poll), next to an offline micro-benchmark of one call on 12 sources. All detectors fit easily inside the 2 s polling interval. XGBoost took 1.540 ms per poll in the controller, and the whole poll took 43.347 ms (95th percentile 104.864 ms) with all seven detectors loaded. The random forest is by far the slowest (200 trees in Python), followed by the SVM and the autoencoder; the threshold and the decision tree are the cheapest. Live times are several times the offline benchmark because seven detectors, the switch handlers and three other experiment containers shared the CPU. With all seven detectors running, the controller used 2.4 % of one CPU core on average during attacks in phase B (maximum run mean 7.4 %) and about 247.0 MB of memory. A separate per-detector CPU measurement was planned (phase O) but dropped for time; since XGBoost's inference uses under 0.1 % of the polling interval, it would not change any conclusion.

**Table 21.** Inference time per poll: measured live inside the controller (phases B + C, polls with at least one source) and offline (median of 300 calls on 12 sources).

| Detector | Live mean (ms) | Live p95 (ms) | Offline, 12 rows (ms) |
|---|---|---|---|
| Static threshold | 0.012 | 0.022 | 0.000 |
| Decision tree | 0.574 | 2.844 | 0.060 |

| Detector | Live mean (ms) | Live p95 (ms) | Offline, 12 rows (ms) |
|---|---|---|---|
| Logistic regression | 1.315 | 5.725 | 0.177 |
| Random forest | 22.294 | 64.009 | 7.250 |
| XGBoost | 1.540 | 5.554 | 0.176 |
| RBF SVM | 3.513 | 12.125 | 0.872 |
| Autoencoder | 3.128 | 11.516 | 0.338 |

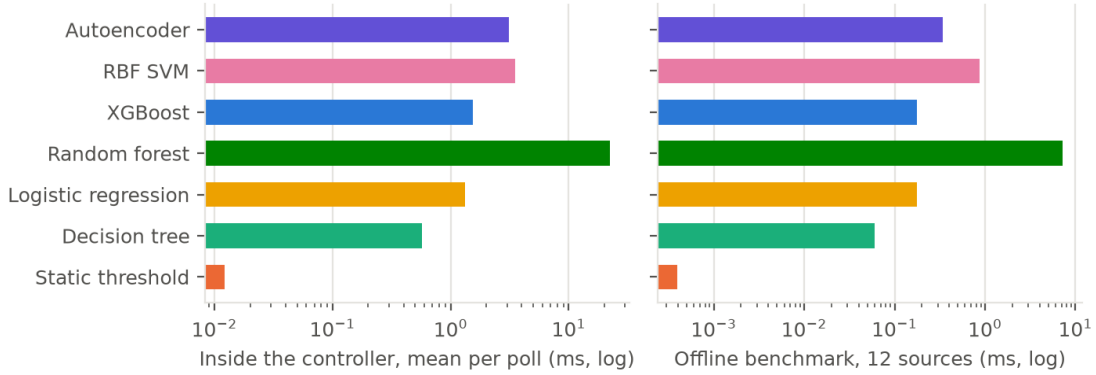



**Figure 11.** Inference time per detector: live in the controller (left) and offline micro-benchmark (right), log scale.

## 8.9 Statistical comparison and where XGBoost does not win

Table 22 pools all phase-B pairs (9 conditions x 5 repetitions) and tests each service metric with a Wilcoxon signed-rank test; Table 30 in Appendix C does the same for phase C. A difference is called significant only when p < 0.05 *and* the bootstrap CI excludes zero.

**Table 22.** Phase B: XGBoost minus baseline, pooled over all condition × repetition pairs.

| Baseline | Metric | n | Mean diff. | 95 % CI | p | Verdict |
|---|---|---|---|---|---|---|
| No defense | TCP goodput / offered | 45 | +0.374 | [+0.254, +0.493] | < 0.001 | XGBoost better |
| No defense | Benign UDP loss (%) | 44 | -16.7 | [-25, -9.32] | < 0.001 | XGBoost better |
| No defense | Ping RTT p50 (ms) | 45 | -212 | [-279, -146] | < 0.001 | XGBoost better |
| No defense | Ping RTT p95 (ms) | 45 | -76.2 | [-106, -48.1] | < 0.001 | XGBoost better |
| No defense | Ping loss (%) | 45 | -17.2 | [-25.6, -9.75] | < 0.001 | XGBoost better |
| No defense | HTTP success (%) | 45 | +13.5 | [+6.58, +21.6] | 0.005 | XGBoost better |
| No defense | HTTP p50 (ms) | 45 | -1.04e+03 | [-1.51e+03, -645] | < 0.001 | XGBoost better |
| No defense | False blocks | 45 | +0 | [+0, +0] | 1.000 | no significant difference |
| No defense | Benign blocked (s) | 45 | +0 | [+0, +0] | 1.000 | no significant difference |
| Static threshold | TCP goodput / offered | 45 | +0.639 | [+0.625, +0.653] | < 0.001 | XGBoost better |
| Static threshold | Benign UDP loss (%) | 44 | +0.722 | [+0.297, +1.23] | 0.002 | XGBoost worse |
| Static threshold | Ping RTT p50 (ms) | 45 | +3.84 | [+3.27, +4.42] | < 0.001 | XGBoost worse |
| Static threshold | Ping RTT p95 (ms) | 45 | -19.7 | [-65.7, +29] | 0.440 | no significant difference |
| Static threshold | Ping loss (%) | 45 | +0.752 | [+0.361, +1.21] | 0.001 | XGBoost worse |
| Static threshold | HTTP success (%) | 45 | +2.61 | [+2.44, +2.79] | < 0.001 | XGBoost better |
| Static threshold | HTTP p50 (ms) | 45 | +12 | [+10.2, +14] | < 0.001 | XGBoost worse |
| Static threshold | False blocks | 45 | -2.38 | [-2.51, -2.24] | < 0.001 | XGBoost better |
| Static threshold | Benign blocked (s) | 45 | -32.7 | [-34, -31.7] | < 0.001 | XGBoost better |
| Static threshold | Time to mitigation (s) | 30 | -0.258 | [-0.482, -0.06] | 0.036 | XGBoost better |
| Decision tree | TCP goodput / offered | 45 | +1.71e-06 | [-4.58e-06, +8.11e-06] | 0.282 | no significant difference |
| Decision tree | Benign UDP loss (%) | 45 | +0.161 | [-0.061, +0.41] | 0.233 | no significant difference |

| Baseline | Metric | n | Mean diff. | 95 % CI | p | Verdict |
|---|---|---|---|---|---|---|
| Decision tree | Ping RTT p50 (ms) | 45 | +0.182 | [-0.211, +0.57] | 0.281 | no significant difference |
| Decision tree | Ping RTT p95 (ms) | 45 | -2.22 | [-20.8, +16.7] | 0.788 | no significant difference |
| Decision tree | Ping loss (%) | 45 | -0.031 | [-0.312, +0.238] | 0.755 | no significant difference |
| Decision tree | HTTP success (%) | 45 | +0.0683 | [-0.0203, +0.177] | 0.173 | no significant difference |
| Decision tree | HTTP p50 (ms) | 45 | +1.66 | [+0.17, +3.21] | 0.063 | no significant difference |
| Decision tree | False blocks | 45 | +0 | [+0, +0] | 1.000 | no significant difference |
| Decision tree | Benign blocked (s) | 45 | +0 | [+0, +0] | 1.000 | no significant difference |
| Decision tree | Time to mitigation (s) | 45 | -0.138 | [-0.287, +0.0148] | 0.031 | no significant difference |

Against no defense, XGBoost is significantly better on every service metric in phase B. Against the threshold the picture is mixed, and we state it plainly. XGBoost is much better on goodput, HTTP success, false blocks, time blocked and time to mitigation. It is slightly *worse* on ping RTT, ping loss, benign UDP loss and HTTP latency, by a few milliseconds or under one percentage point. The threshold wins these because it has blocked the benign iperf3 host, which frees bottleneck capacity for the remaining probes; the price is about two thirds of that host's goodput. Against the decision tree no difference is significant: on this traffic a depth-6 tree captures nearly everything XGBoost does. Per condition, 59 paired comparisons came out significantly worse for XGBoost; 55 of them are against the threshold (HTTP p50 (ms) (18); Ping RTT p50 (ms) (16); Ping RTT p95 (ms) (12); Benign UDP loss (%) (4); Ping loss (%) (3); Time to mitigation (s) (2)), and the remaining 4 are listed in Table 23. With several hundred per-condition tests, a few small differences against the decision tree are expected by chance.

**Table 23.** Conditions where XGBoost was worse than no defense or the decision tree (bootstrap CI of the paired difference excludes zero on the unfavourable side; n = 5 pairs). Cases against the threshold are summarised in the text.

| Phase | Condition | Baseline | Metric | Mean diff. | 95 % CI |
|---|---|---|---|---|---|
| B | icmp-low | Decision tree | Ping RTT p50 (ms) | +0.903 | [+0.04, +1.77] |
| B | syn-low | Decision tree | Ping RTT p95 (ms) | +3.11 | [+0.492, +5.51] |
| B | icmp-medium | Decision tree | HTTP p50 (ms) | +2.53 | [+0.25, +4.81] |
| C | udp-wan | No defense | Benign UDP loss (%) | +0.232 | [+0.0386, +0.406] |

## 8.10 CICDDoS2019: what the feature mismatch costs

Table 24 and Figure 12 report the CICDDoS2019 experiments. With all CICFlowMeter features (FULL), every tree model is almost perfect, whether the split is random or by capture day. This is consistent with the well-known separability of this dataset and should not be read as evidence about live SDN detection. Restricting the models to OpenFlow-computable features (OF) changes little on a random split (XGBoost F1 0.997). Trained on one day and tested on the other, however, XGBoost's F1 drops to 0.850 (recall 0.743). The detectors generalise across capture sessions far worse than the random split suggests, and the feature mismatch makes this worse.

**Table 24.** CICDDoS2019 sample: binary detection (attack = positive). FULL = all CICFlowMeter features; OF = OpenFlow-computable forward-flow features.

| Experiment | Model | F1 | Recall | FPR % | Bal. acc. | AUC |
|---|---|---|---|---|---|---|
| FULL, random 70/30 (day 01-12) | Decision tree | 0.9994 | 0.9988 | 0.06 | 0.9991 | 0.9995 |
| FULL, random 70/30 (day 01-12) | Logistic regression | 0.9989 | 0.9986 | 0.56 | 0.9965 | 0.9997 |
| FULL, random 70/30 (day 01-12) | Random forest | 0.9998 | 0.9997 | 0.06 | 0.9995 | 1.0000 |

| Experiment | Model | F1 | Recall | FPR % | Bal. acc. | AUC |
|---|---|---|---|---|---|---|
| FULL, random 70/30 (day 01-12) | XGBoost | 0.9999 | 0.9999 | 0.09 | 0.9995 | 1.0000 |
| FULL, train 01-12 / test 03-11 | Decision tree | 0.9997 | 0.9996 | 0.14 | 0.9991 | 0.9995 |
| FULL, train 01-12 / test 03-11 | Logistic regression | 0.9987 | 0.9990 | 0.87 | 0.9952 | 0.9997 |
| FULL, train 01-12 / test 03-11 | Random forest | 0.9884 | 0.9788 | 0.90 | 0.9849 | 0.9985 |
| FULL, train 01-12 / test 03-11 | XGBoost | 0.9975 | 0.9998 | 2.66 | 0.9866 | 1.0000 |
| OF, random 70/30 (day 01-12) | Decision tree | 0.9957 | 0.9958 | 3.51 | 0.9803 | 0.9966 |
| OF, random 70/30 (day 01-12) | Logistic regression | 0.8937 | 0.8148 | 7.07 | 0.8721 | 0.9596 |
| OF, random 70/30 (day 01-12) | Random forest | 0.9974 | 0.9990 | 3.44 | 0.9823 | 0.9972 |
| OF, random 70/30 (day 01-12) | XGBoost | 0.9975 | 0.9993 | 3.61 | 0.9816 | 0.9972 |
| OF, random 70/30 (day 01-12) | Static threshold | 0.9429 | 1.0000 | 100.00 | 0.5000 | 0.7676 |
| OF, train 01-12 / test 03-11 | Decision tree | 0.8500 | 0.7433 | 3.07 | 0.8563 | 0.7819 |
| OF, train 01-12 / test 03-11 | Logistic regression | 0.8379 | 0.7423 | 15.84 | 0.7919 | 0.9284 |
| OF, train 01-12 / test 03-11 | Random forest | 0.8498 | 0.7434 | 3.26 | 0.8554 | 0.9009 |
| OF, train 01-12 / test 03-11 | XGBoost | 0.8496 | 0.7434 | 3.55 | 0.8539 | 0.9607 |
| OF, train 01-12 / test 03-11 | Static threshold | 0.9149 | 1.0000 | 100.00 | 0.5000 | 0.7635 |

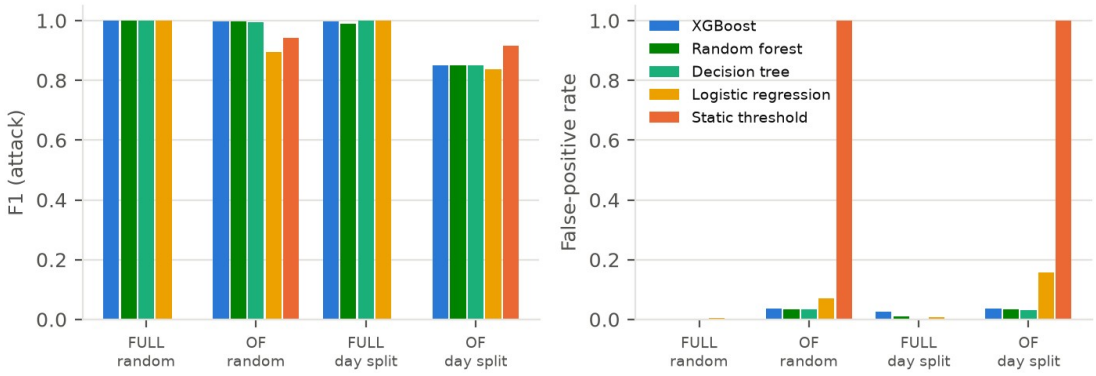



**Figure 12.** CICDDoS2019: F1 and false-positive rate for FULL vs OF features under random and day splits.

The tuned threshold on the OF features degenerates to flagging every flow (FPR 100 %). Attacks are about 85 % of the sample, so flagging everything already gives F1 above 0.9. This is a known weakness of F1 on attack-majority data, which is why Table 24 also reports balanced accuracy. Finally, Table 25 tests transfer between the two data sources using the same OF flow features. Mininet flow records come from OpenFlow flow-removed messages, and a flow counts as attack if it is the attacker's flood protocol and overlaps the attack window. Their durations are corrected for the 10 s idle timeout, but flows still active at the end of a run (the long-lived ping and iperf3 streams) never produce a flow-removed message and are missing, so the Mininet benign class under-represents heavy benign flows. Transfer is poor and inconsistent. From CICDDoS2019 to Mininet, balanced accuracy ranges from chance (logistic regression) to 0.833 (random forest), with XGBoost at 0.685. From Mininet to CICDDoS2019 every trained model is at chance (0.500 for XGBoost). The in-domain Mininet-to-Mininet reference reaches 0.912. The Mininet test set is small (Table 25), so these numbers are indicative only. CICDDoS2019 is dominated by reflection attacks on a different network, so it is not a substitute for training data collected in the deployment network.

**Table 25.** Cross-domain transfer with OpenFlow-computable flow features (balanced accuracy is the fairest single number here).

| Train → test | Model | Test flows | F1 | Recall | FPR % | Bal. acc. |
|---|---|---|---|---|---|---|
| CICDDoS2019 (OF) → Mininet phase B | Decision tree | 233 | 0.667 | 1.000 | 62.9 | 0.685 |
| CICDDoS2019 (OF) → Mininet phase B | Logistic regression | 233 | 0.000 | 0.000 | 0.0 | 0.500 |
| CICDDoS2019 (OF) → Mininet phase B | Random forest | 233 | 0.800 | 0.667 | 0.0 | 0.833 |
| CICDDoS2019 (OF) → Mininet phase B | XGBoost | 233 | 0.667 | 1.000 | 62.9 | 0.685 |
| CICDDoS2019 (OF) → Mininet phase B | Static threshold | 233 | 0.557 | 1.000 | 100.0 | 0.500 |
| Mininet phase A → CICDDoS2019 (OF) | Decision tree | 889321 | 0.000 | 0.000 | 0.0 | 0.500 |
| Mininet phase A → CICDDoS2019 (OF) | Logistic regression | 889321 | 0.074 | 0.038 | 1.8 | 0.510 |
| Mininet phase A → CICDDoS2019 (OF) | Random forest | 889321 | 0.000 | 0.000 | 0.0 | 0.500 |
| Mininet phase A → CICDDoS2019 (OF) | XGBoost | 889321 | 0.000 | 0.000 | 0.0 | 0.500 |
| Mininet phase A → CICDDoS2019 (OF) | Static threshold | 889321 | 0.879 | 0.864 | 69.0 | 0.587 |
| Mininet A → Mininet B (reference) | Decision tree | 233 | 0.857 | 0.900 | 12.6 | 0.887 |
| Mininet A → Mininet B (reference) | Logistic regression | 233 | 0.769 | 0.833 | 21.0 | 0.812 |
| Mininet A → Mininet B (reference) | Random forest | 233 | 0.862 | 0.900 | 11.9 | 0.891 |
| Mininet A → Mininet B (reference) | XGBoost | 233 | 0.888 | 0.922 | 9.8 | 0.912 |
| Mininet A → Mininet B (reference) | Static threshold | 233 | 0.796 | 1.000 | 32.2 | 0.839 |

### 8.11 Ablations

- **Feature set (CICDDoS2019):** FULL vs OF, Section 8.10. The OF restriction is what a controller-based deployment actually faces.
- **Confirmation intervals:** requiring two consecutive positive intervals instead of one adds about 2 s to the block latency (1.23 s vs 3.23 s for XGBoost, Table 10). For the trained models it removes nothing they would otherwise catch (0 % missed attackers), and it protects against isolated false positives.
- **Threshold value:** Figure 6. Even the best threshold chosen with hindsight on the test data stays far below the trained models.
- **Detector family:** seven detectors on identical traffic (Table 10). Most of the benefit comes from using a trained classifier at all; the choice among tree models matters much less.

## 9. Discussion

### 9.1 What worked and what did not

**Worked.** A controller-only pipeline built from OpenFlow counters was enough to detect all three flood types at all three rates, block attackers about 3.19 s after they start, and restore benign service, with about 1.540 ms of XGBoost inference per poll. Shadow evaluation made every comparison paired and cheap. Reading the drop rule's counters for the unblock decision behaved as intended: attackers stayed blocked while they flooded and were released a few seconds after they stopped.

**Did not work, or worked less than hoped.** (i) The improvement of the proposed XGBoost over a plain decision tree is tiny in detection and not significant in the closed loop, and a random forest detected marginally better; the honest headline is *trained tree models beat conventional rate thresholds by a wide margin*, not *XGBoost is special*. (ii) The unsupervised autoencoder failed

at its a-priori threshold. (iii) Models trained on CICDDoS2019 did not transfer to our network, and even within CICDDoS2019 the OpenFlow feature set generalised poorly across days. (iv) Some detectors rely on packet size and protocol mix; this is effective against hping3 floods but is an evasion opportunity.

## 9.2 Threats to validity and limitations

- **Synthetic traffic (external validity).** Benign traffic is generated by a small set of programs (HTTP clients, ping, iperf3) and attacks by hping3 with fixed packet sizes. Real benign traffic is more diverse, and real attackers can randomise sizes and protocols. The near-perfect F1 of the trained models should be read as *on this traffic*.
- **Same generator for training and test.** Phase A and phases B/C use different seeds and different condition samples, but the same traffic generators. This is a weaker separation than training and testing on different networks; the CICDDoS2019 transfer results (Section 8.10) show how large that gap can be.
- **No spoofing, no reflection.** Per-source detection assumes attackers use their own addresses. A spoofed-source flood would create one flow per fake source and defeat per-source features (and stress the packet-in path); reflection/amplification attacks, which dominate CICDDoS2019, were not emulated.
- **Labels.** An attacker's interval counts as attack if it overlaps the attack window at all, including its benign browsing during the attack. Edge intervals are therefore slightly ambiguous; this affects all detectors equally.
- **One-factor-at-a-time design.** Phase C varies one factor at a time from a default; interactions (e.g. high load *and* WAN loss) are untested.
- **Label boundaries.** An attacker's interval counts as attack if it overlaps the attack window at all, so the first and last attack intervals contain partly benign traffic; conversely, packets queued at the bottleneck reach the switch for up to about 1 s after hping3 stops. This mostly affects detection latency and edge-interval recall, equally for all detectors.
- **Conditional time to mitigation.** TTM averages only attackers that were blocked; a detector that misses low-rate attackers (the threshold) therefore has fewer TTM samples, and the share of attackers blocked must be read alongside it.
- **Measurement repairs.** The review also found that iperf3 occasionally wrote diagnostic text around its JSON when it was stopped at the end of a run; the parser now extracts the JSON object, recovering every affected run (mostly undefended and threshold runs, whose loss would otherwise have biased comparisons in XGBoost's favour). Ping send times are interpolated between received probes, and false blocks are judged against each run's ground-truth attack windows rather than host addresses.
- **Small n per condition.** Five repetitions give wide per-condition intervals; significance claims are made only on pooled tests.
- **Parallel execution.** Phases A–C ran three or four containers at once on a 20-thread host. Run order was shuffled and concurrent runs achieved the same attack rates as solo runs (RUNLOG D-17), but timing and CPU measurements may carry some extra noise.
- **Emulation fidelity.** Mininet shares one kernel and CPU among hosts; link rates are enforced by HTB, and the high rate achieved about 95 % of its nominal value (Table 8).

- **Dataset provenance.** CICDDoS2019 was taken from a public mirror (hashes recorded) and sub-sampled; known CICFlowMeter issues [22]–[24] may affect its labels.
- **Controller scope.** A single controller instance; controller-targeted attacks (packet-in floods) and controller failover were not studied.

### 9.3 Review of the original repository

The project began from an early prototype. Its critical review (REVIEW.md) found that the repository did not match the project brief: there was no controller application, detector, mitigation, label or attack trace, and the brief's planned files did not exist. The most important defects, all fixed (CHANGELOG.md), were:

**Table 26.** Highest-severity findings of the repository review and their fixes.

| ID | Finding (evidence from the committed data) | Fix |
|---|---|---|
| C2 | Flows matched only MAC addresses, so no per-source IP feature could be computed; the feature CSV had src_ip = UNKNOWN in every row | IP-level forwarding rules; ingress-switch attribution |
| C3 | Cumulative counters were written as per-window values: consecutive windows were identical | Flow-delta tracker with unit tests |
| C4 | No labels and no attack ever recorded (maximum rate 10.4 pps) | Orchestrator ground truth on the controller clock |
| H1 | Rates divided by a constant 5 s while real poll gaps were 4–6 s (up to ±20 % error) | Measured interval |
| H2/H3 | Counter resets clamped to zero, losing traffic of re-created flows; flows never expired | Restart detection + FlowRemoved; idle timeouts |
| H4 | Switch totals double/triple counted multi-hop packets | Ingress-switch-only features |
| H6 | Segmentation offload left on: packet-size feature read 52–53 kB per 'packet' | Offload disabled on all hosts |
| H7 | Sessions appended into one file without run IDs (leakage risk) | One directory and manifest per run |
| M1 | Ryu 4.34 fails on current Python | os-ken 4.2.2 in a pinned container |

## 10. Conclusion and future work

We built a working SDN DDoS defense on Mininet, Open vSwitch and os-ken, and evaluated it in 447 live runs instead of on a static dataset alone. Using only features an OpenFlow 1.3 switch can provide, a tree-based detector found SYN, UDP and ICMP floods from 50 to 5 000 pps per attacker. Its drop rules restored benign goodput, latency and HTTP success under every tested network condition except where random link loss itself limited TCP, at a cost of about 1.540 ms of inference per polling interval. Against a tuned static rate threshold the improvement is large and statistically clear. The threshold misses low-rate floods and blocks legitimate high-rate senders, and no threshold value fixes both. Against a simple decision tree the improvement is negligible, and a random forest detected marginally better; we report it as such. The CICDDoS2019 experiments add a caution for the literature: near-perfect offline accuracy on full CICFlowMeter features says little about a controller, which sees fewer features, a different network and different attacks.

**Future work.** (1) Evasion-aware evaluation: attackers that randomise packet size and protocol, low-and-slow attacks, and spoofed-source floods with port- or aggregate-level features. (2) Rate limiting with OpenFlow meters instead of hard drops, to soften the cost of false positives. (3)

Federated training across several SDN domains, following [4], [5], [7], so that models learn from attacks seen elsewhere without sharing raw traffic. (4) Operator-facing explanations that turn the logged SHAP attributions into short text, as in [10]. (5) Factorial designs for interactions between network conditions, and tests on real traffic traces replayed into Mininet.

# References


[1] R. Carillo, F. Cerasuolo, G. Bovenzi, D. Ciuonzo, and A. Pescapè, "Explainable federated class incremental learning for encrypted network traffic classification," *Comput. Netw.*, vol. 269, Art. no. 111448, Sep. 2025, doi: 10.1016/j.comnet.2025.111448.

[2] R. Kalakoti, S. Nõmm, and H. Bahsi, "Federated learning of explainable AI (FedXAI) for deep learning-based intrusion detection in IoT networks," *Comput. Netw.*, vol. 270, Art. no. 111479, Oct. 2025, doi: 10.1016/j.comnet.2025.111479.

[3] J. P. Sahoo, B. Kar, A. M. Abdelmoniem, and D. Chatzopoulos, "Choir-IDS: A federated learning framework for fidelity-calibrated explainable intrusion detection system for edge-IoT networks," *Inf. Fusion*, vol. 125, Art. no. 103473, Jan. 2026, doi: 10.1016/j.inffus.2025.103473.

[4] J. Ma and W. Su, "Collaborative DDoS defense for SDN-based AIoT with autoencoder-enhanced federated learning," *Inf. Fusion*, vol. 117, Art. no. 102820, May 2025, doi: 10.1016/j.inffus.2024.102820.

[5] A. Tellache, A. Amara Korba, A. Mokhtari, and Y. Ghamri-Doudane, "Collaborative privacy-preserving network intrusion detection: A federated multi-agent reinforcement learning approach," *Comput. Commun.*, vol. 256, Art. no. 108565, Jul. 2026, doi: 10.1016/j.comcom.2026.108565.

[6] W. Wang, Y. Liu, Q. Meng, and Z. Chen, "DDoS attack detection and defense techniques in software defined networks: A survey," *Comput. Sci. Rev.*, vol. 60, Art. no. 100921, May 2026, doi: 10.1016/j.cosrev.2026.100921.

[7] D. Cajaraville-Aboy *et al.*, "CO-DEFEND: Continuous decentralized federated learning for secure DoH-based threat detection," *Comput. Netw.*, vol. 276, Art. no. 111961, Feb. 2026, doi: 10.1016/j.comnet.2025.111961.

[8] S. Li, R. Du, J. Chen, K. He, C. Wu, and Y. Feng, "CANDICE: An explainable and intelligent framework for network intrusion detection," *Future Gener. Comput. Syst.*, vol. 175, Art. no. 108059, Feb. 2026, doi: 10.1016/j.future.2025.108059.

[9] C. Minh, K. Vermeulen, C. Lefebvre, P. Owezarski, and W. Ritchie, "When the weakest model sees the threat: An explainable ensemble learning system for detecting network attacks," *Comput. Netw.*, vol. 273, Art. no. 111730, Dec. 2025, doi: 10.1016/j.comnet.2025.111730.

[10] P. R. B. Houssel, S. Layeghy, P. Singh, and M. Portmann, "eX-NIDS: A framework for explainable network intrusion detection leveraging large language models," *Comput. Electr. Eng.*, vol. 129, Art. no. 110826, Jan. 2026, doi: 10.1016/j.compeleceng.2025.110826.

[11] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Chennai, India, Oct. 2019, pp. 1–8, doi: 10.1109/CCST.2019.8888419.

[12] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop Hot Topics Netw. (HotNets-IX)*, Monterey, CA, USA, Oct. 2010, pp. 1–6, doi: 10.1145/1868447.1868466.

[13] N. McKeown *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Apr. 2008, doi: 10.1145/1355734.1355746.

[14] Open Networking Foundation, "OpenFlow switch specification, version 1.3.0 (wire protocol 0x04)," ONF TS-006, Jun. 2012. [Online]. Available: https://opennetworking.org/wp-content/uploads/2014/10/openflow-spec-v1.3.0.pdf

[15] Ryu SDN Framework Community, "Ryu SDN framework documentation." [Online]. Available: https://ryu.readthedocs.io/en/latest/ (accessed Oct. 28, 2026).

[16] OpenStack, "os-ken: A component-based software defined networking framework for OpenStack," version 4.2.2. [Online]. Available: https://docs.openstack.org/os-ken/latest/ (accessed Oct. 28, 2026).

[17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Red Hook, NY, USA: Curran Associates, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[18] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.

[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[20] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[21] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[22] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, San Francisco, CA, USA, May 2021, pp. 7–12, doi: 10.1109/SPW53761.2021.00009.

[23] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, LNCS, vol. 13857. Cham, Switzerland: Springer, 2023, pp. 18–33, doi: 10.1007/978-3-031-31108-6_2.

[24] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, 2022, pp. 254–262, doi: 10.1109/CNS56114.2022.9947235.

[25] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proc. IEEE 35th Conf. Local Comput. Netw. (LCN)*, Denver, CO, USA, Oct. 2010, pp. 408–415, doi: 10.1109/LCN.2010.5735752.

[26] L. Tan, Y. Pan, J. Wu, J. Zhou, H. Jiang, and Y. Deng, "A new framework for DDoS attack detection and defense in SDN environment," *IEEE Access*, vol. 8, pp. 161908–161919, 2020, doi: 10.1109/ACCESS.2020.3021435.

[27] N. Ahuja, G. Singal, D. Mukhopadhyay, and N. Kumar, "Automated DDOS attack detection in software defined networking," *J. Netw. Comput. Appl.*, vol. 187, Art. no. 103108, Aug. 2021, doi: 10.1016/j.jnca.2021.103108.

# Appendix A. Reproduction and repository structure

Every run directory (results/runs/<phase>/<run_id>/) contains spec.json (parameters and seed), manifest.json (ground-truth attack start/end per attacker and the run's start time t0), the controller's features.csv (all features and all detector scores per interval), events.csv (blocks/unblocks with SHAP attributions), timing.csv, flows_removed.csv, resources.csv, and the raw iperf3, ping, HTTP and hping3 logs. The commands below rebuild everything from scratch on a Linux host with Docker.

```
docker build -f docker/Dockerfile -t sdnddos:dev .
scripts/in_container.sh mn --test pingall
scripts/analysis.sh -m pytest -q tests
python3 experiments/plan.py A && scripts/run_phase.sh A 3
scripts/analysis.sh analysis/build_dataset.py A && scripts/analysis.sh analysis/train.py
for p in B C; do python3 experiments/plan.py $p --proposed xgb && scripts/run_phase.sh $p 4;
done
for p in B C; do scripts/analysis.sh analysis/build_dataset.py $p; done
scripts/analysis.sh analysis/eval_detection.py B C
scripts/analysis.sh analysis/eval_mitigation.py B C
scripts/analysis.sh analysis/explain_offline.py
scripts/analysis.sh analysis/cic_experiments.py cic
scripts/analysis.sh analysis/cic_experiments.py xdom
scripts/analysis.sh analysis/plots.py
scripts/analysis.sh report/facts.py && node report/build_report.js
```

```
sdnddos/        controller_app.py run_controller.py flowstats.py features.py detectors.py
                mitigation.py explain.py labels.py topo.py traffic.py benign_client.py
orchestrator.py config.py
experiments/    plan.py run_phase.py
analysis/       build_dataset.py train.py eval_detection.py eval_mitigation.py run_metrics.py
                explain_offline.py cic_prepare.py cic_experiments.py stats.py plots.py
scripts/        in_container.sh analysis.sh run_phase.sh
tests/          test_core.py (25 unit tests)
docker/         Dockerfile entrypoint.sh
results/        runs/ datasets/ tables/ figures/ logs/
models/         detectors.joblib train_report.json
report/         facts.py facts_results.py build_report.js chapters_*.js lib.js
legacy/         original prototype (unchanged)
```

**Figure 13.** Repository structure.

# Appendix B. Parameters

**Table 27.** Full parameter list.

| Parameter | Value |
|---|---|
| Polling interval | 2 s (flow statistics request to every switch) |
| Forwarding rule | priority 10, match eth_type/ipv4_src/ipv4_dst/ip_proto, idle timeout 10 s, SEND_FLOW_REM |
| Drop rule | priority 100, match ipv4_src, no actions, no timeout (controller-managed), on the ingress switch |
| Mitigation | confirm 2 intervals; min block 10 s, doubled per repeat offence; unblock after drop-rule pps < 20 for 2 polls; server IP protected |
| Threshold detector | pps ≥ 171 (max-F1 on phase A) |
| Decision tree | max_depth 6, class_weight balanced |
| Logistic regression | signed log1p + standardisation, class_weight balanced, max_iter 2000 |
| RBF SVM | signed log1p + standardisation, C = 1, class_weight balanced, Platt calibration (CalibratedClassifierCV, ensemble = False) |

| Parameter | Value |
|---|---|
| Random forest | 200 trees, min_samples_leaf 2, class_weight balanced |
| XGBoost | 300 trees, max_depth 4, learning rate 0.1, subsample 0.9, colsample_bytree 0.9 |
| Autoencoder | MLP 10-8-3-8-10 on signed-log standardised features, benign rows only; threshold = 99th percentile of benign error (2.25) |
| Proba threshold | 0.5 for all probabilistic detectors |
| Topologies | small: 1 switch, 3 benign + 3 attackers + server; large: 2 switches, 8 benign + 4 attackers + server |
| Links | default 10 Mb/s, 2 ms, 0 %; bw50 50 Mb/s; wan 10 Mb/s, 20 ms, 1 % loss; trunk 4 x host bandwidth |
| Benign load | HTTP think time mean 2 s / 0.5 s / 0.1 s (low/medium/high); attackers 2 s; files 1/10/100 kB (p = 0.5/0.3/0.2) |
| Probes | ping every 0.2 s (b1); iperf3 TCP at 0.4·B (b2); iperf3 UDP at 0.1·B (b3) |
| Attacks | hping3; SYN: -S -p 80; UDP: --udp -p 9 -d 512; ICMP: --icmp -d 512; rates 50/500/5 000 pps per attacker |
| Run timing | phase A: 10 s warm-up, 30 s attack, 10 s tail; phases B/C: 10/30/15 s |
| Seeds | phase A 1000+i; B 5000+10·c+r; C 7000+10·c+r; run order 42; training 7; CIC sampling 2019; SHAP sample 3 |

# Appendix C. Additional results

**Table 28.** Benign UDP (iperf3, 0.1·B) loss during the attack, % (phase B). Mean ± 95 % CI half-width, n = 5.

| Condition | No defense | Threshold | Decision tree | XGBoost |
|---|---|---|---|---|
| SYN low | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 |
| SYN medium | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 |
| SYN high | 24.7 ±15.8 | 1.4 ±1.5 | 2.6 ±0.9 | 2.6 ±1.1 |
| UDP low | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 |
| UDP medium | 0.5 ±0.3 | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 |
| UDP high | 73.5 ±1.4 | 6.8 ±2.7 | 9.7 ±1.1 | 10.5 ±1.3 |
| ICMP low | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 |
| ICMP medium | 0.6 ±0.1 | 0.0 ±0.0 | 0.0 ±0.0 | 0.0 ±0.0 |
| ICMP high | 75.3 ±3.9 | 7.7 ±0.3 | 9.2 ±1.8 | 9.8 ±2.5 |

**Table 29.** Detection under phase-C conditions (mode none): F1 and FPR (%).

| Condition | F1 XGB | DT | Thr. | FPR % XGB | DT | Thr. |
|---|---|---|---|---|---|---|
| SYN, high benign load | 1.000 | 0.982 | 0.545 | 0.00 | 1.32 | 58.36 |
| SYN, 50 Mb/s links | 1.000 | 1.000 | 0.528 | 0.00 | 0.00 | 60.75 |
| SYN, WAN: 20 ms, 1 % loss | 0.994 | 0.992 | 0.780 | 0.00 | 0.14 | 17.30 |
| SYN, large topology | 0.998 | 0.998 | 0.690 | 0.00 | 0.00 | 19.40 |
| UDP, high benign load | 1.000 | 0.998 | 0.604 | 0.00 | 0.14 | 41.29 |
| UDP, 50 Mb/s links | 1.000 | 1.000 | 0.519 | 0.00 | 0.00 | 60.00 |
| UDP, WAN: 20 ms, 1 % loss | 0.990 | 0.990 | 0.830 | 0.42 | 0.42 | 11.06 |
| UDP, large topology | 1.000 | 0.998 | 0.755 | 0.00 | 0.07 | 12.41 |
| ICMP, high benign load | 1.000 | 0.990 | 0.584 | 0.00 | 0.72 | 44.14 |
| ICMP, 50 Mb/s links | 0.998 | 0.998 | 0.521 | 0.00 | 0.00 | 60.11 |
| ICMP, WAN: 20 ms, 1 % loss | 1.000 | 0.998 | 0.777 | 0.00 | 0.00 | 17.33 |
| ICMP, large topology | 0.998 | 1.000 | 0.722 | 0.00 | 0.00 | 15.18 |
| Flash crowd (benign only) | — | — | — | 0.00 | 0.70 | 46.72 |

**Table 30.** Phase C: XGBoost minus baseline, pooled over all condition × repetition pairs (including the flash crowd).

| Baseline | Metric | n | Mean diff. | 95 % CI | p | Verdict |
|---|---|---|---|---|---|---|
| No defense | TCP goodput / offered | 65 | +0.172 | [+0.108, +0.242] | < 0.001 | XGBoost better |
| No defense | Benign UDP loss (%) | 62 | -0.35 | [-0.527, -0.191] | < 0.001 | XGBoost better |
| No defense | Ping RTT p50 (ms) | 65 | -142 | [-192, -91.8] | < 0.001 | XGBoost better |
| No defense | Ping RTT p95 (ms) | 65 | -40.6 | [-58.2, -24.8] | < 0.001 | XGBoost better |
| No defense | Ping loss (%) | 65 | -0.497 | [-0.807, -0.197] | 0.003 | XGBoost better |
| No defense | HTTP success (%) | 65 | +0 | [+0, +0] | 1.000 | no significant difference |
| No defense | HTTP p50 (ms) | 65 | -283 | [-384, -184] | < 0.001 | XGBoost better |
| No defense | False blocks | 65 | +0 | [+0, +0] | 1.000 | no significant difference |
| No defense | Benign blocked (s) | 65 | +0 | [+0, +0] | 1.000 | no significant difference |
| Static threshold | TCP goodput / offered | 65 | +0.519 | [+0.455, +0.578] | < 0.001 | XGBoost better |
| Static threshold | Benign UDP loss (%) | 28 | +0.211 | [+0.0493, +0.372] | 0.021 | XGBoost worse |
| Static threshold | Ping RTT p50 (ms) | 65 | +4.35 | [+3.32, +5.4] | < 0.001 | XGBoost worse |
| Static threshold | Ping RTT p95 (ms) | 65 | +87.4 | [+49.1, +126] | < 0.001 | XGBoost worse |
| Static threshold | Ping loss (%) | 65 | -0.623 | [-1.71, +0.134] | 0.455 | no significant difference |
| Static threshold | HTTP success (%) | 65 | +3.97 | [+2.96, +5.03] | < 0.001 | XGBoost better |
| Static threshold | HTTP p50 (ms) | 65 | +9.27 | [+7.08, +11.5] | < 0.001 | XGBoost worse |
| Static threshold | False blocks | 65 | -2.89 | [-3.26, -2.52] | < 0.001 | XGBoost better |
| Static threshold | Benign blocked (s) | 65 | -60.5 | [-69.7, -51.1] | < 0.001 | XGBoost better |
| Static threshold | Time to mitigation (s) | 60 | -0.426 | [-0.621, -0.233] | < 0.001 | XGBoost better |

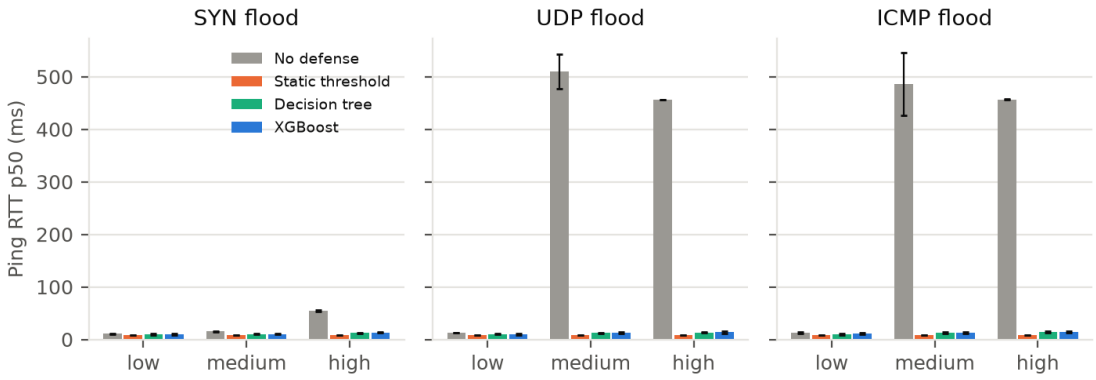



**Figure 14.** Median ping RTT during the attack (phase B).

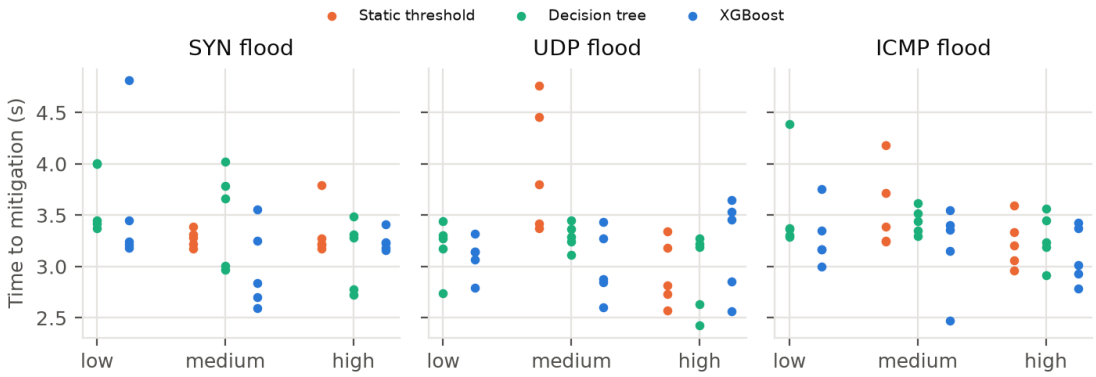



**Figure 15.** Time to mitigation per run (mean over attackers), by mode and condition (phase B). Missing points: no attacker blocked.

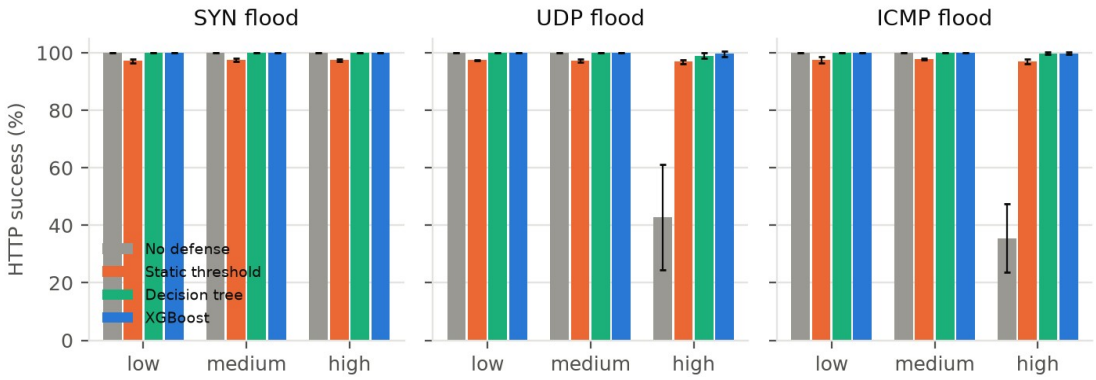



**Figure 16.** Benign HTTP success rate during the attack (phase B).

**Live vs offline scoring.** For every detector, re-scoring the logged phase B/C features offline gave at most 0 decision mismatches over 23,220 rows (table live_vs_offline.csv). Logged features are rounded to six significant digits, which moves tree-ensemble scores slightly near split points without changing decisions.